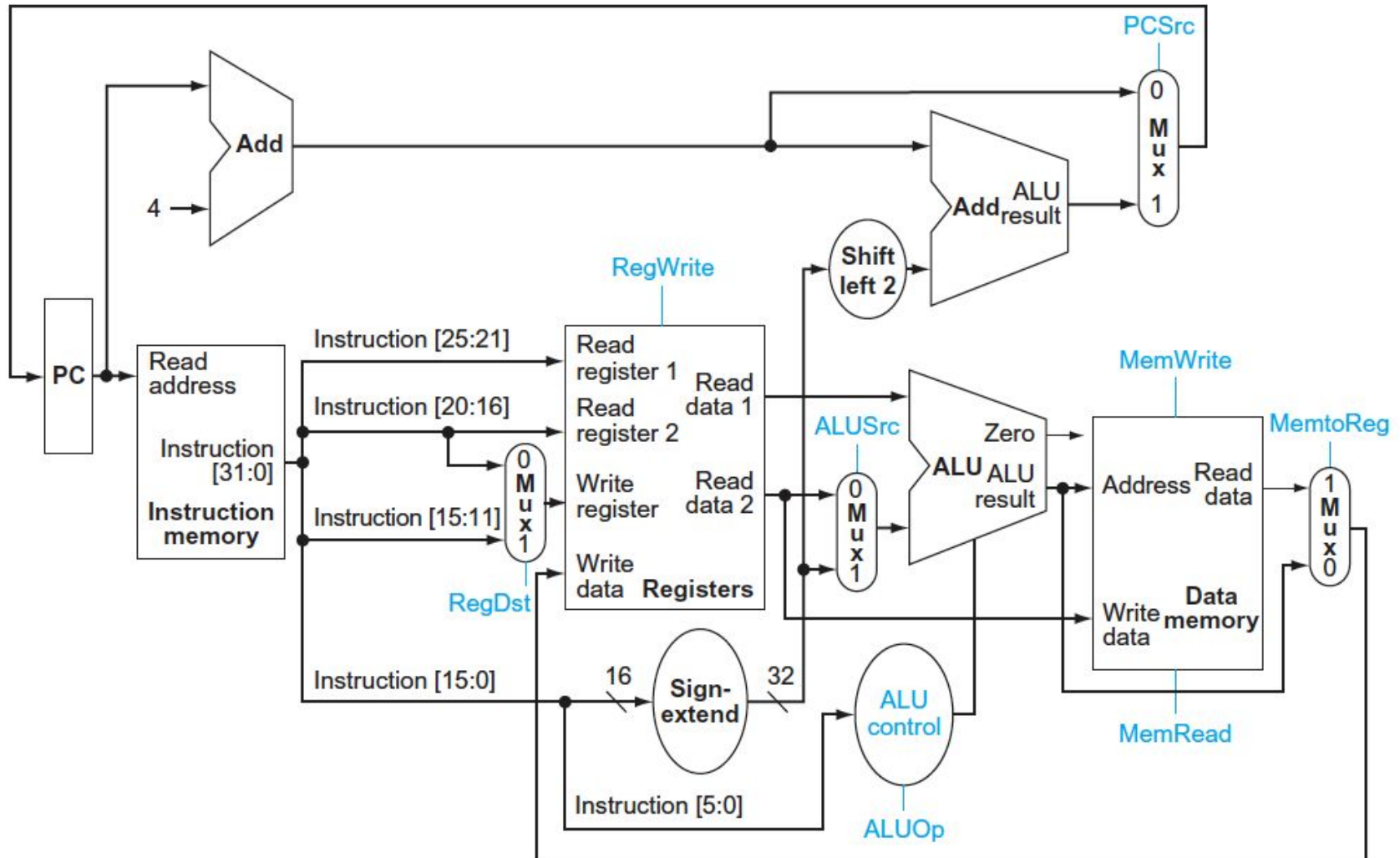


Arquitetura de Computadores

Tópico 3 - Organização MIPS Multiciclo

Todas as instruções usam todo HW?



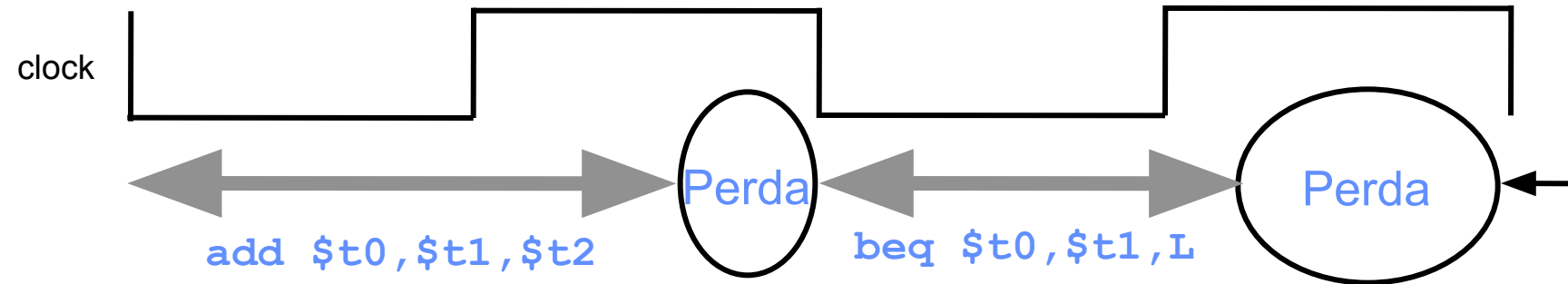
Organização Monociclo



- Todas as instruções completam em um ciclo de relógio ($CPI = 1$)
- Algumas instruções se propagam por mais HW que outras!
 - LW propaga por 5 grande módulos de HW
 - Tipo R e SW propaga por 4 grandes módulos de HW
 - BEQ propaga por 3 grandes módulos de HW
- O relógio deve garantir a execução da instrução mais longa
 - Ineficiência!

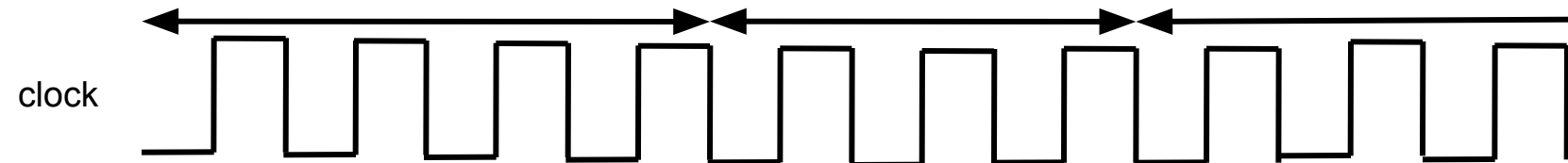
Clock: Mono vs. Multiciclo

Mono-Ciclo



add \$t0,\$t1,\$t2

beq \$t0,\$t1,L



Multiciclo

Organização Multiciclo

- Queremos uma implementação mais eficiente
- Cada ***passo*** levará ***um ciclo de relógio***
 - Não mais cada instrução, como no Monociclo
 - Um passo corresponde a execução de uma parte do hardware
- Menor caminho crítico
 - **O ciclo de relógio é restrito pelo passo mais longo**, não mais pela instrução mais longa como no Monociclo
 - Instruções que se propagam por menos hardware levam menos ciclos
- Continua-se a ter somente **UMA** instrução dentro do processador

Organização Multiciclo

- Qual o caminho crítico da organização multiciclo?
 - Depende do **passo mais demorado**
 - Como medir isto?
 - Verificando o atraso das unidades funcionais de cada passo
- Como dividir as instruções em passos?
 - Inserir registradores para limitar a propagação da instrução no caminho combinacional
- O CPI ficará maior que 1?
 - Sim
 - Note que o CPI será dependente da aplicação
 - Dependerá da quantidade de instruções de cada tipo que o código possui

Passos das Instruções

1. Busca da instrução na memória/Incremento do PC

2. Decodificação da instrução

Leitura dos registradores – mesmo que não sejam utilizados

Cálculo do endereço de salto do branch – mesmo que instrução não seja branch

3. *(dependente do tipo da instrução)*

- *Tipo R* – Execução da operação
- *Load e Store* – Cálculo do endereço do operando na memória
- *Branch* – Determinar se branch deve ser executado; atualiza PC (*acaba*)
- *Jump* – Atualiza o conteúdo do PC (*acaba*)

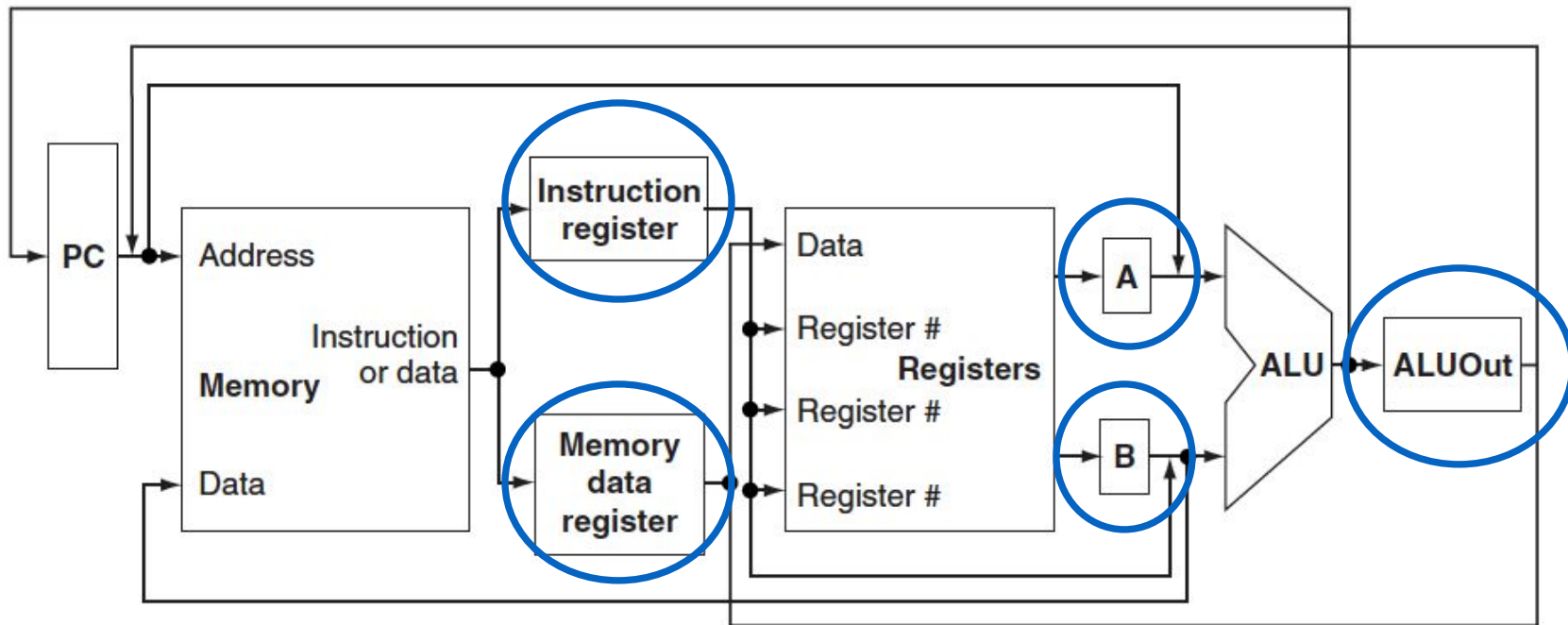
4. *(dependente do tipo da instrução)*

- *Tipo R* – Grava resultado no banco de registradores (*acaba*)
- *Store* – Grava dado na memória (*acaba*)
- *Load* – Busca dado na memória

• 5. *Load* – Grava resultado no banco de registradores (*acaba*)

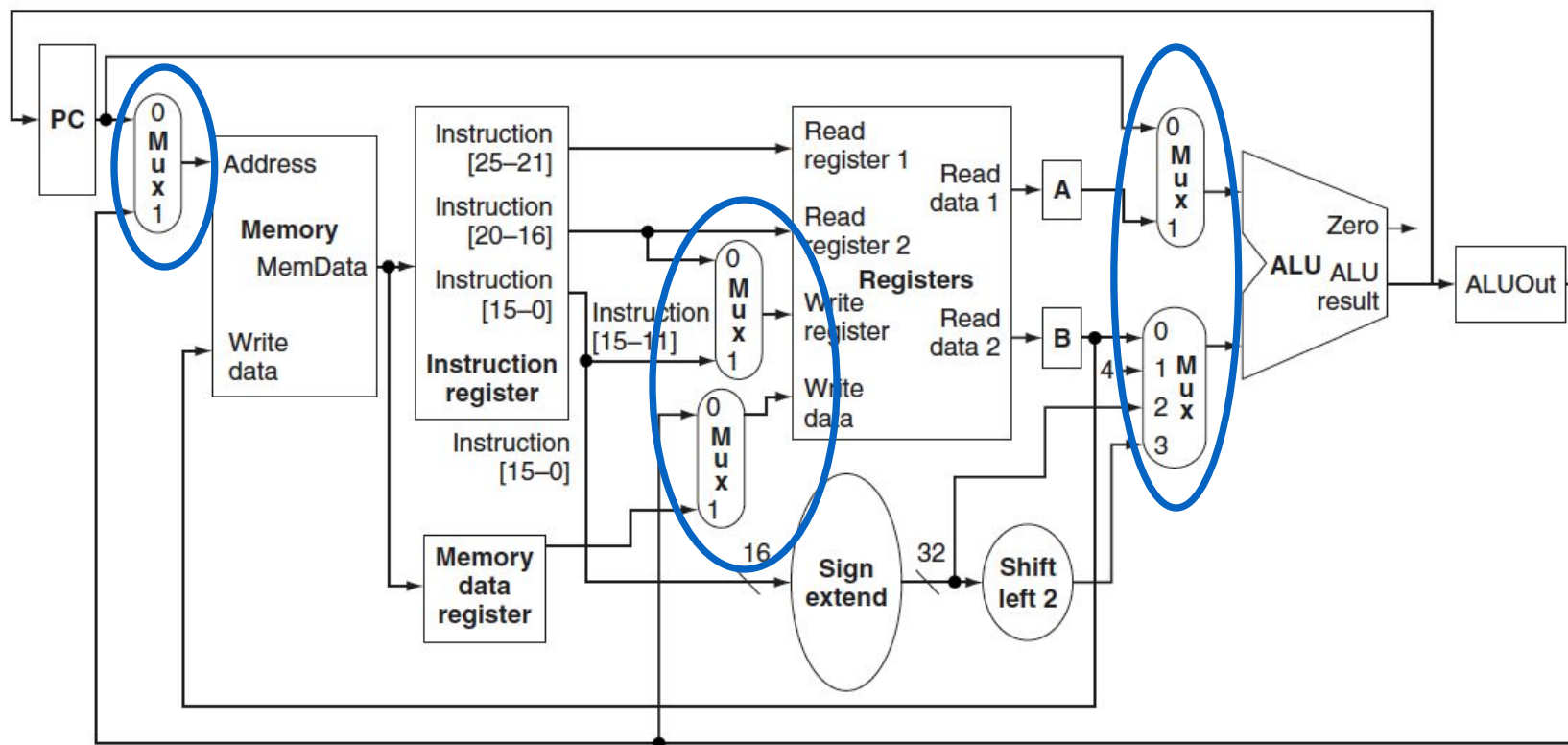
Por que não podem ser feitos no mesmo passo?

Bloco operacional



Novos registradores

Bloco operacional



Novos multiplexadores

Comparando com o Monociclo

- Uma única memória para dados e instruções
- Uma única unidade lógica e aritmética para todas as operações
 - operações das instruções tipo R
 - cálculo de $PC = PC + 4$
 - cálculo de endereço efetivo de memória: base + deslocamento
 - cálculo de endereço de desvio: $PC + \text{deslocamento}$
- Novos registradores
 - Registrador de Instruções
 - Registrador de Dados da Memória (MDR)
 - A e B: guardam valores dos operandos do banco de registradores
 - ALU out: guarda valor da saída da ALU

Resumidamente:

- **Máquina mono-ciclo**

- todas as instruções devem ser feitas em um ciclo
- duração do ciclo calculada pela instrução que leva mais tempo
- leitura da instrução e acesso à memória dados no mesmo ciclo: duas memórias
- cálculos de endereço e operações aritméticas no mesmo ciclo: três unidades funcionais (ALU, somadores)

- **Máquina multi-ciclo**

- vários ciclos por instrução
- cada instrução pode ser executada num número diferente de ciclos
- unidades funcionais podem ser **reutilizadas** em ciclos distintos

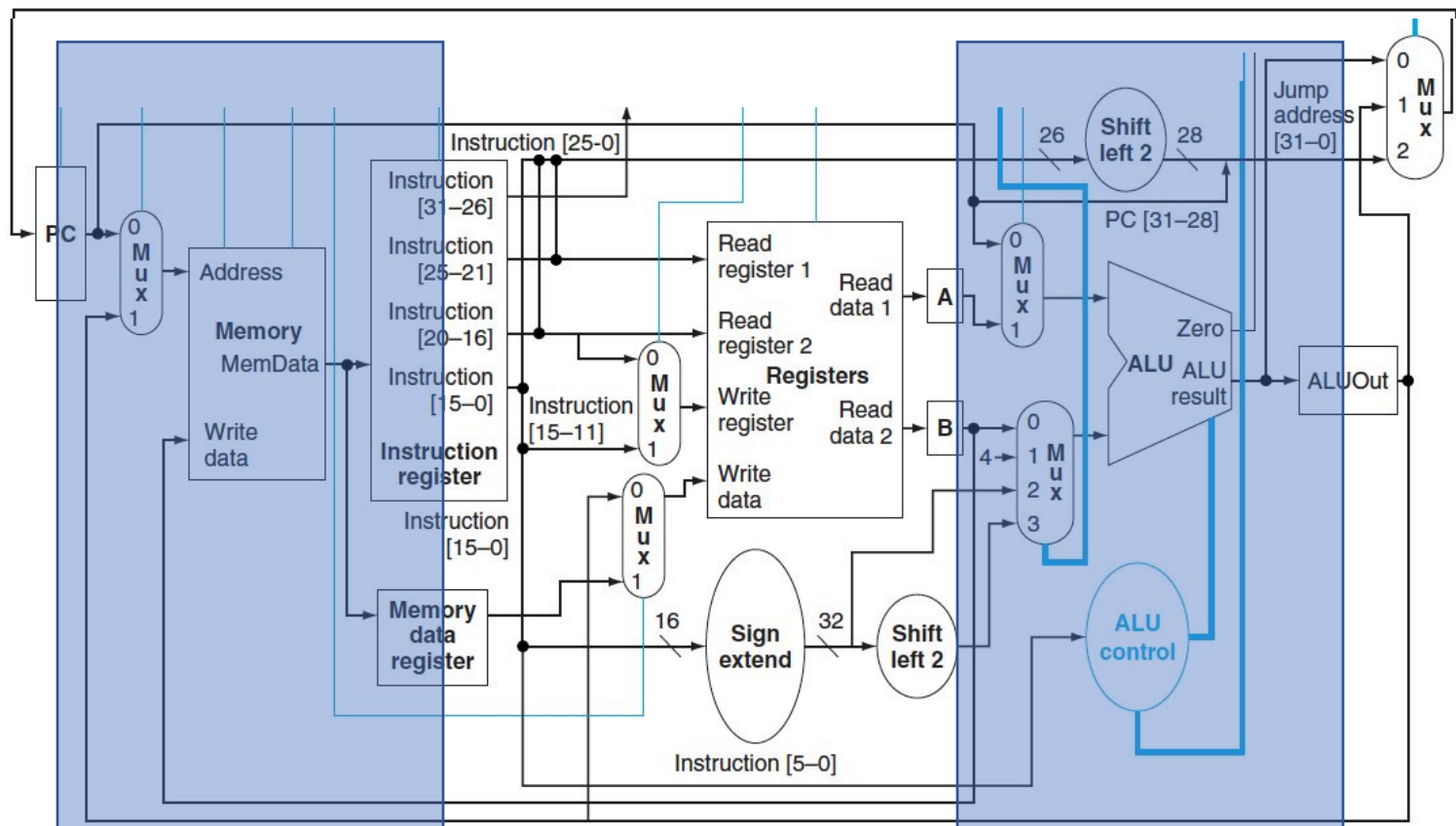
- **Multi-ciclo versus Mono-Ciclo**

- CPI aumenta => desempenho cai
- **MAS** período do relógio diminui => desempenho sobe

Todas as Instruções

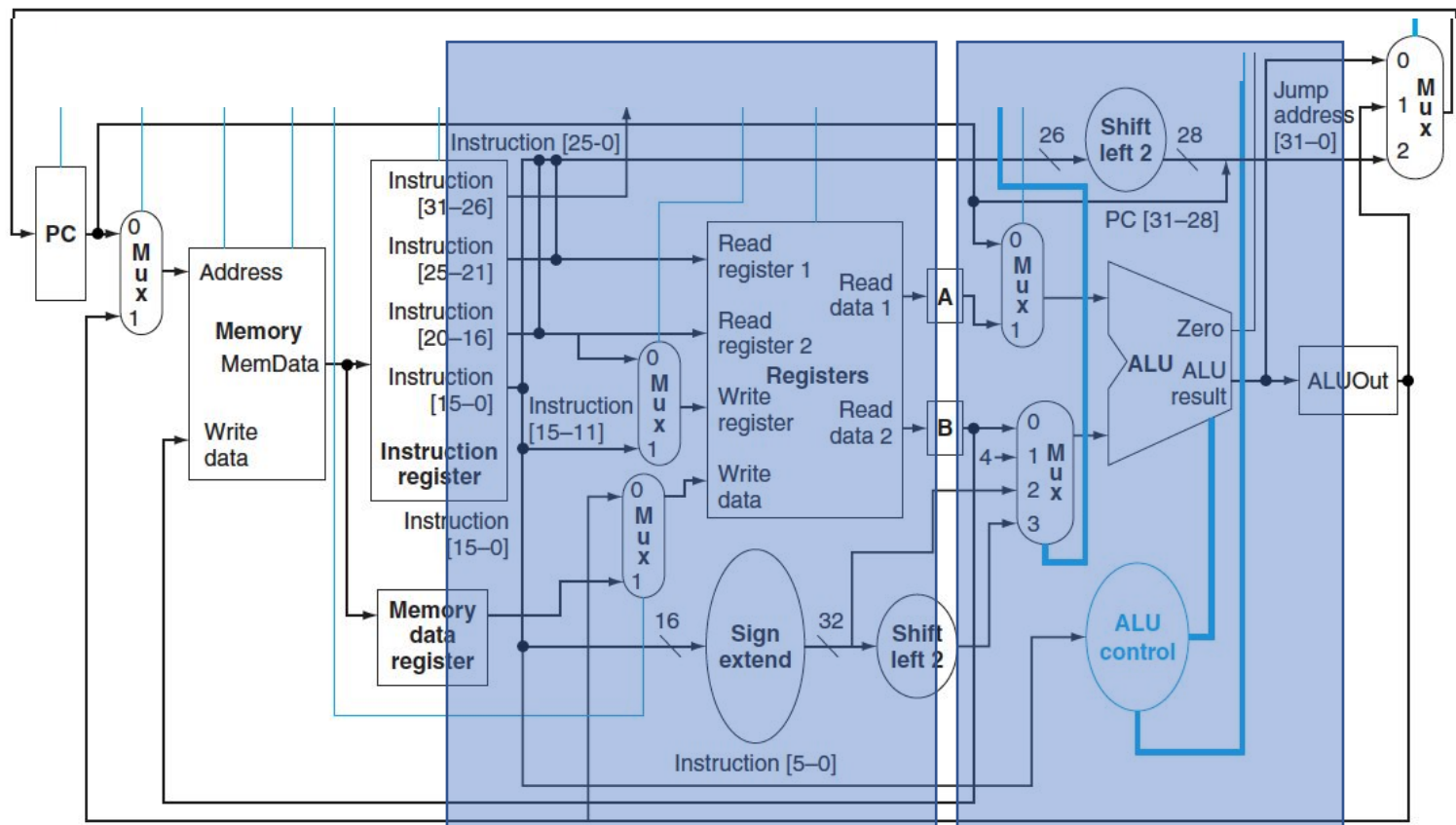
Passo 1 – Busca de Instruções

```
IR = Memory[PC];  
PC = PC + 4;
```



Passo 2 – Dec/Ler BR/Calc Branch

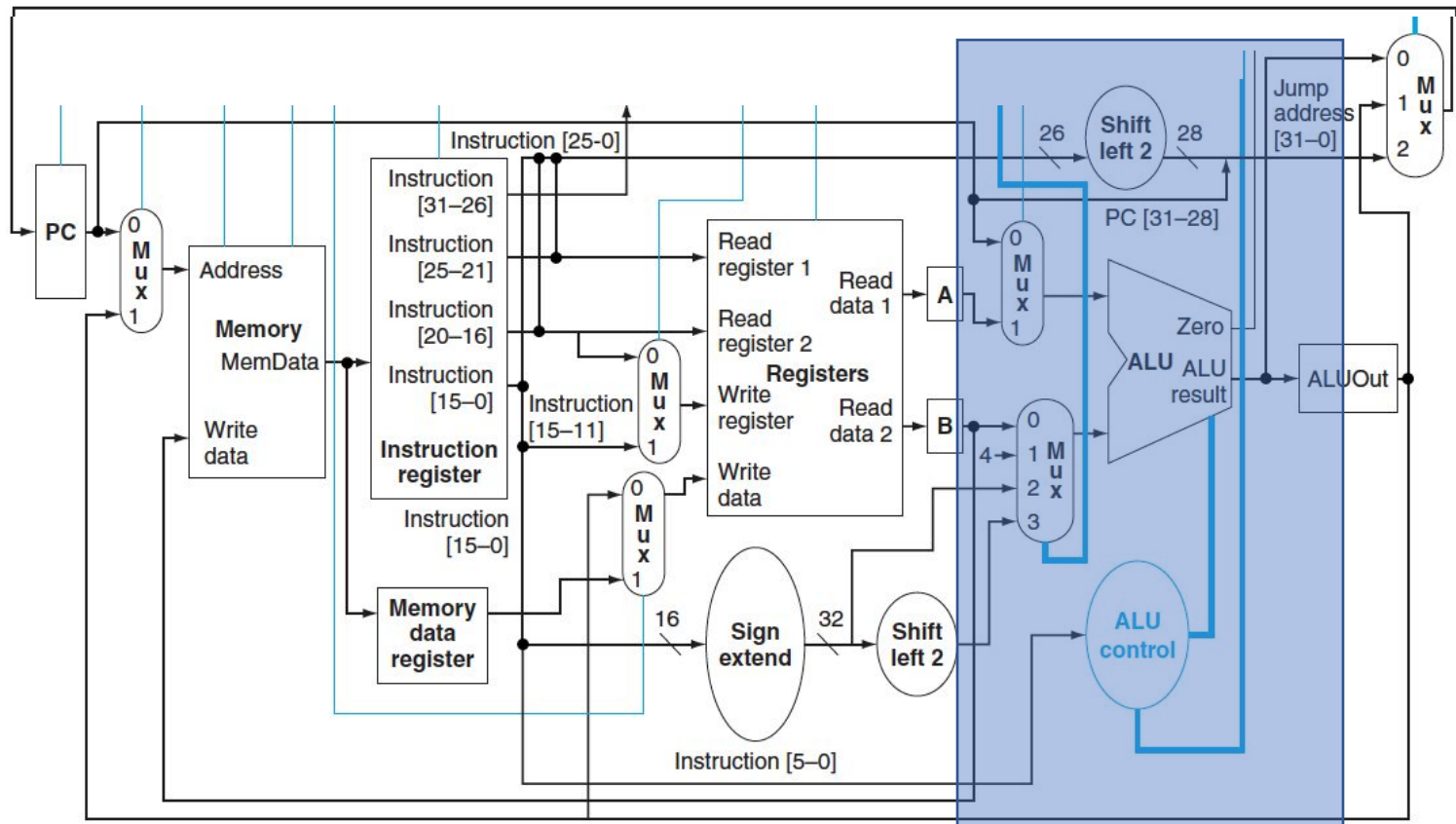
```
A = Reg[IR[25-21]];      (A = Reg[rs])  
B = Reg[IR[20-15]];      (B = Reg[rt])  
ALUOut = (PC + sign-extend(IR[15-0]) << 2)
```



Instruções Tipo R

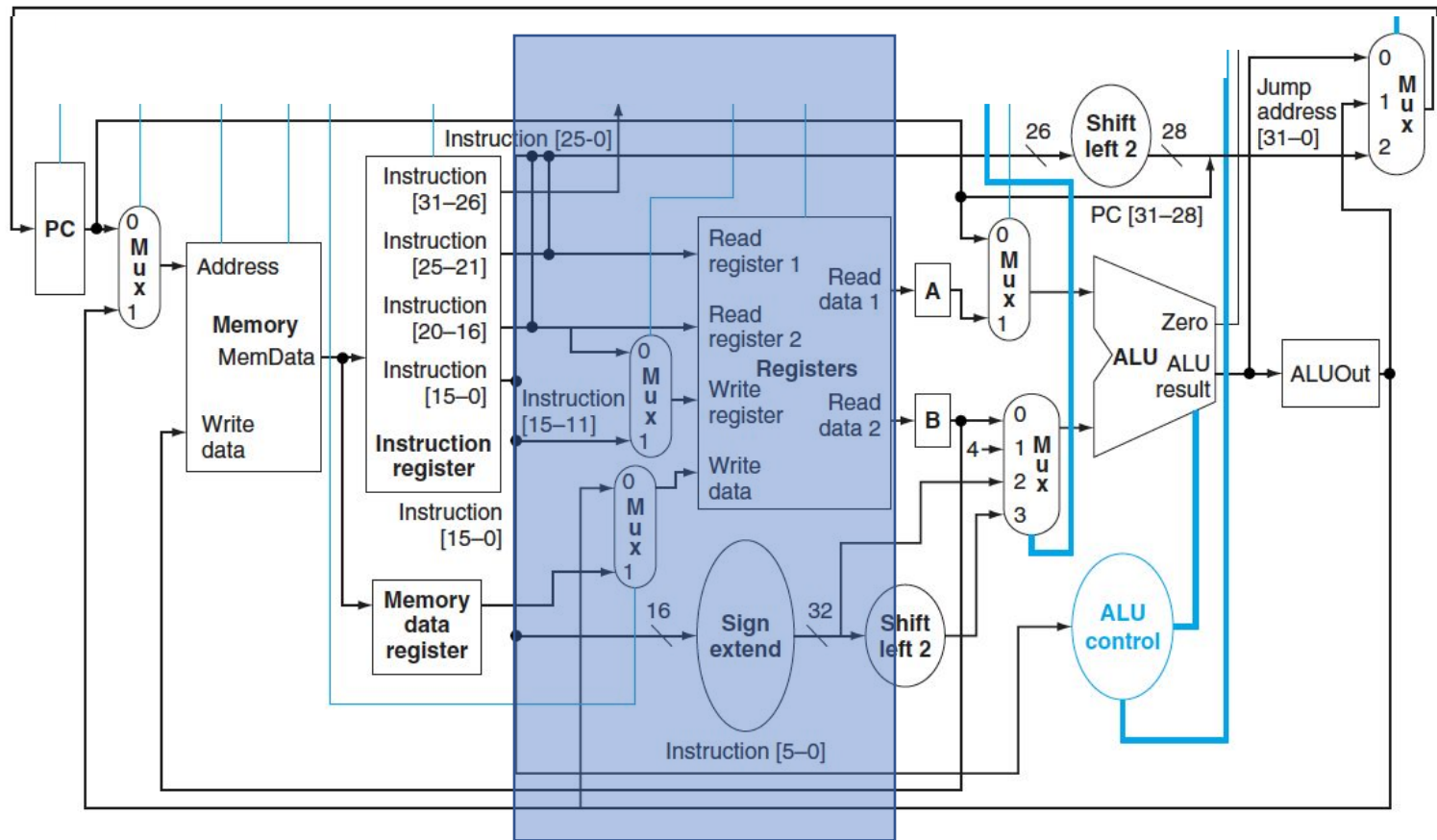
Passo 3 – Instruções tipo R

$$\text{ALUOut} = A \text{ op } B$$



Passo 4 – Instruções tipo R

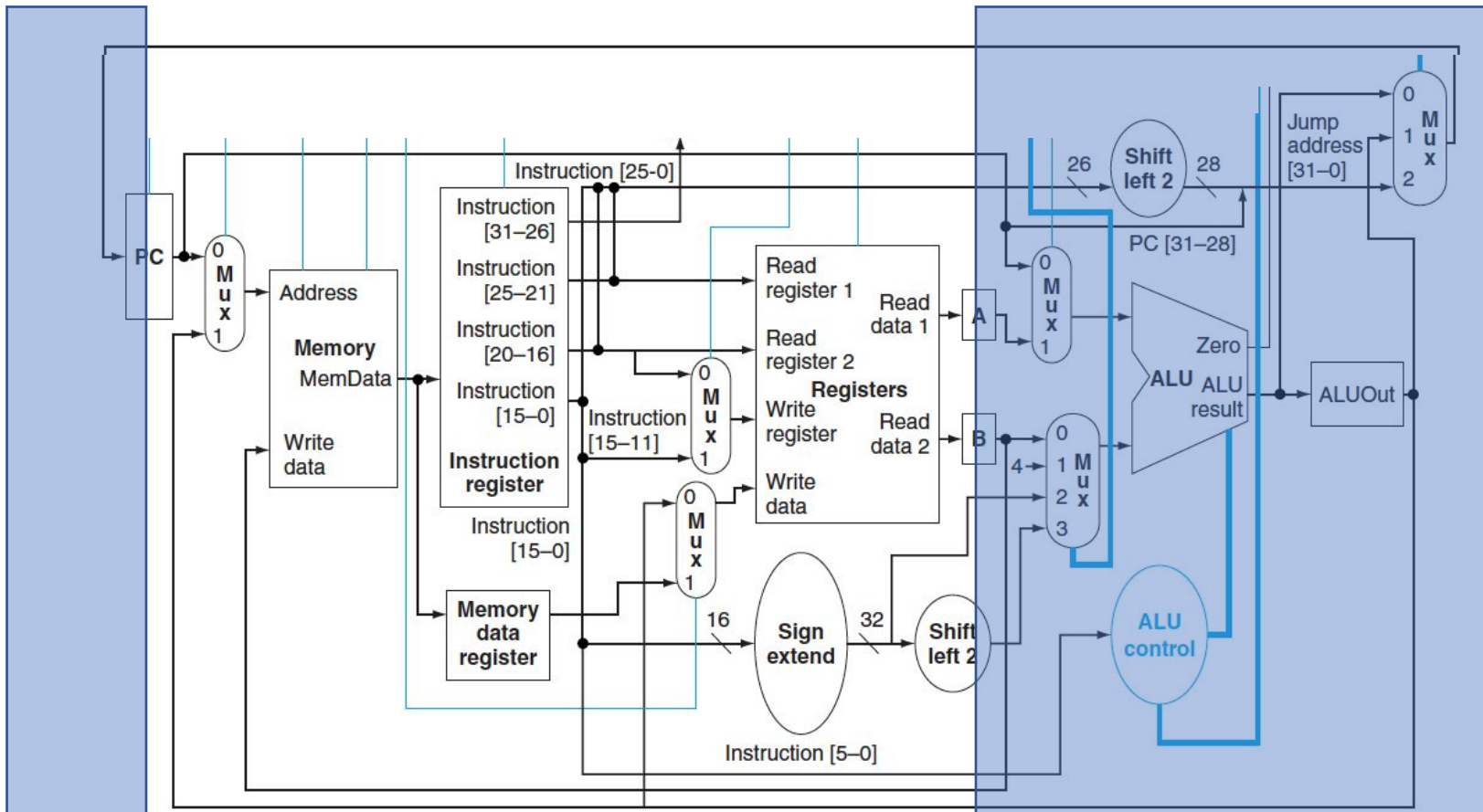
$\text{Reg}[\text{IR}[15:11]] = \text{ALUOUT}$



Instruções Branch

Passo 3 - Branch

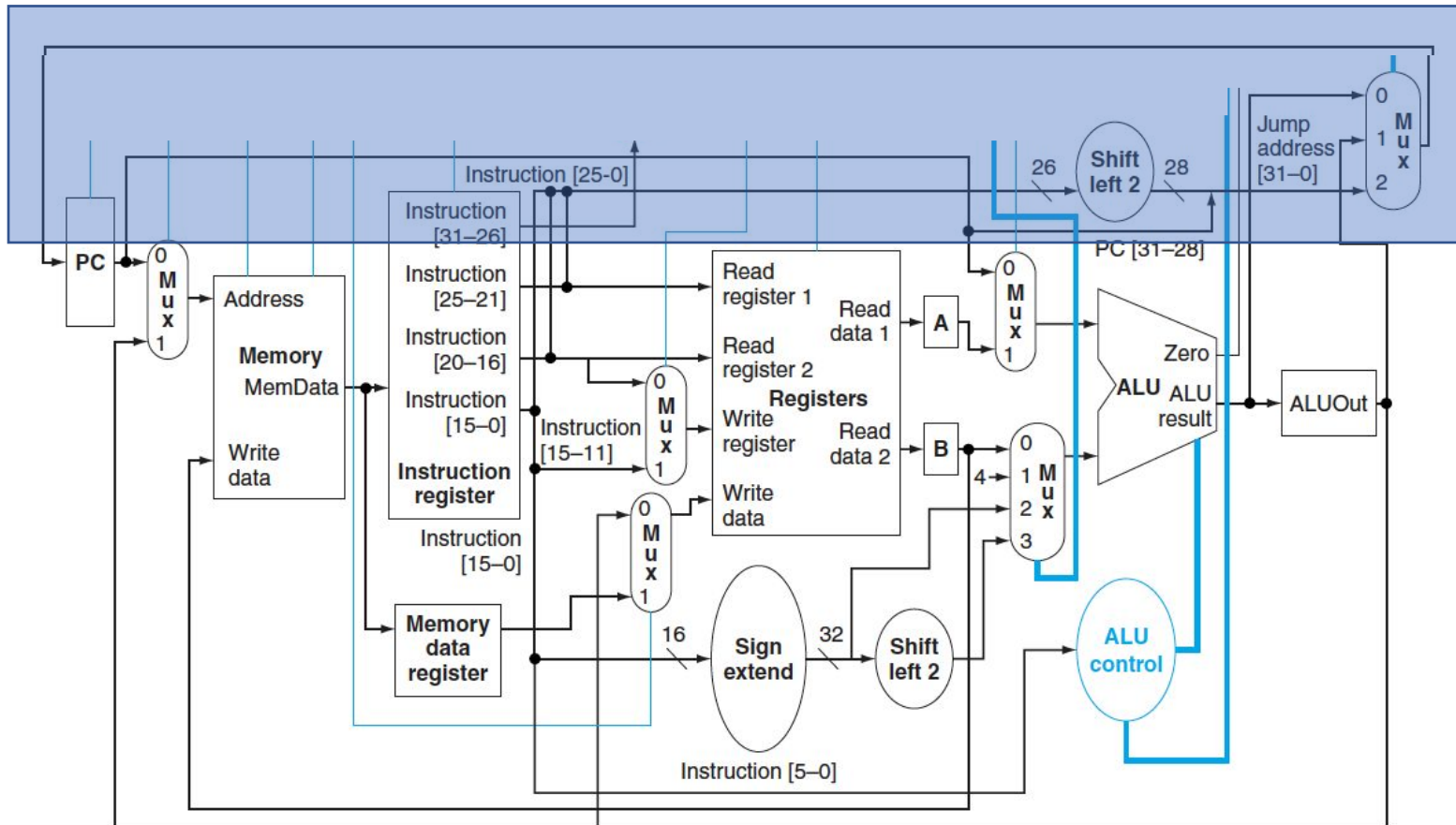
```
if (A == B) PC = ALUOut;
```



Instrução Jump

Passo 3 - Jump

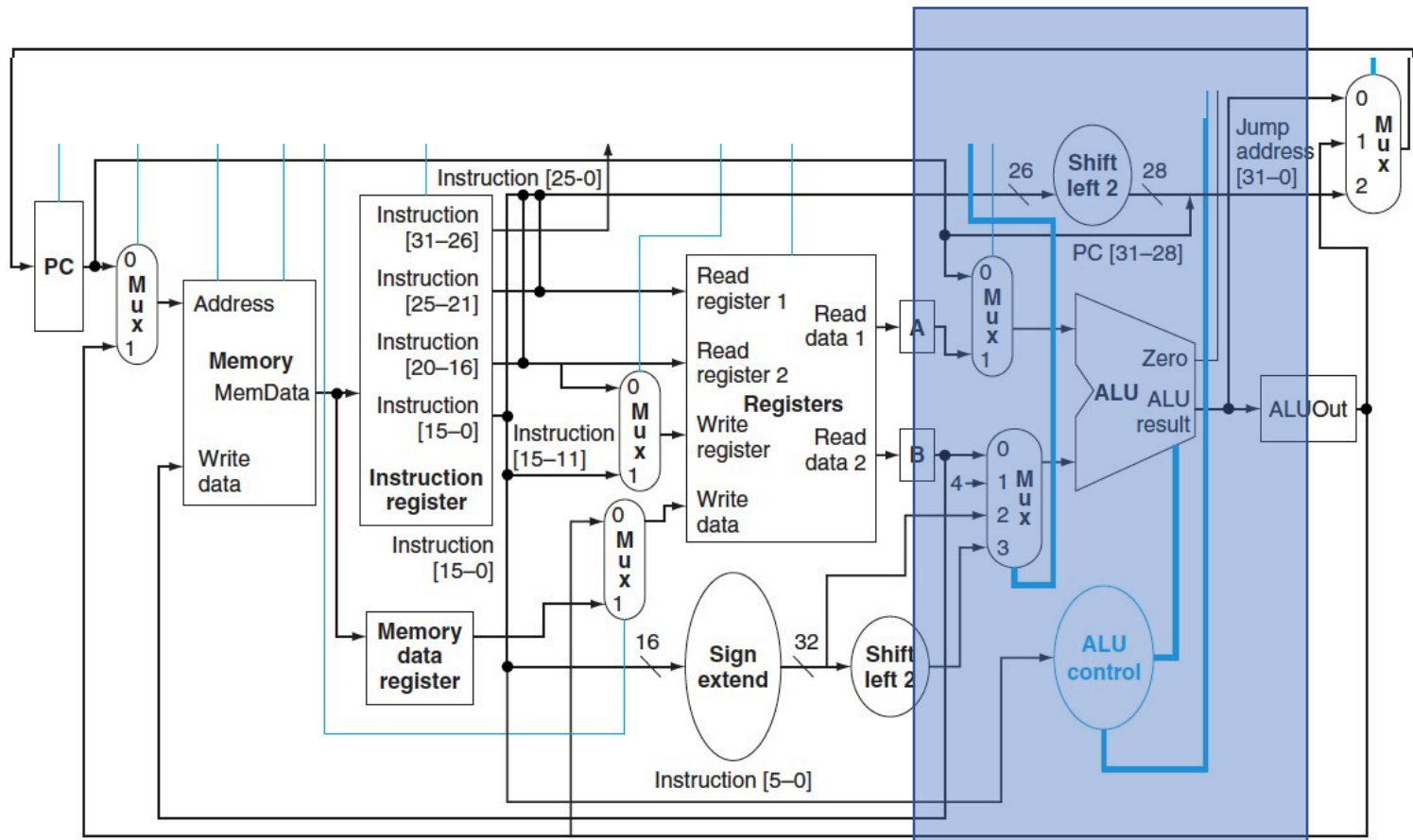
$PC = PC[31-28] \text{ concat } (IR[25-0] \ll 2)$



Instruções de Acesso à Memória

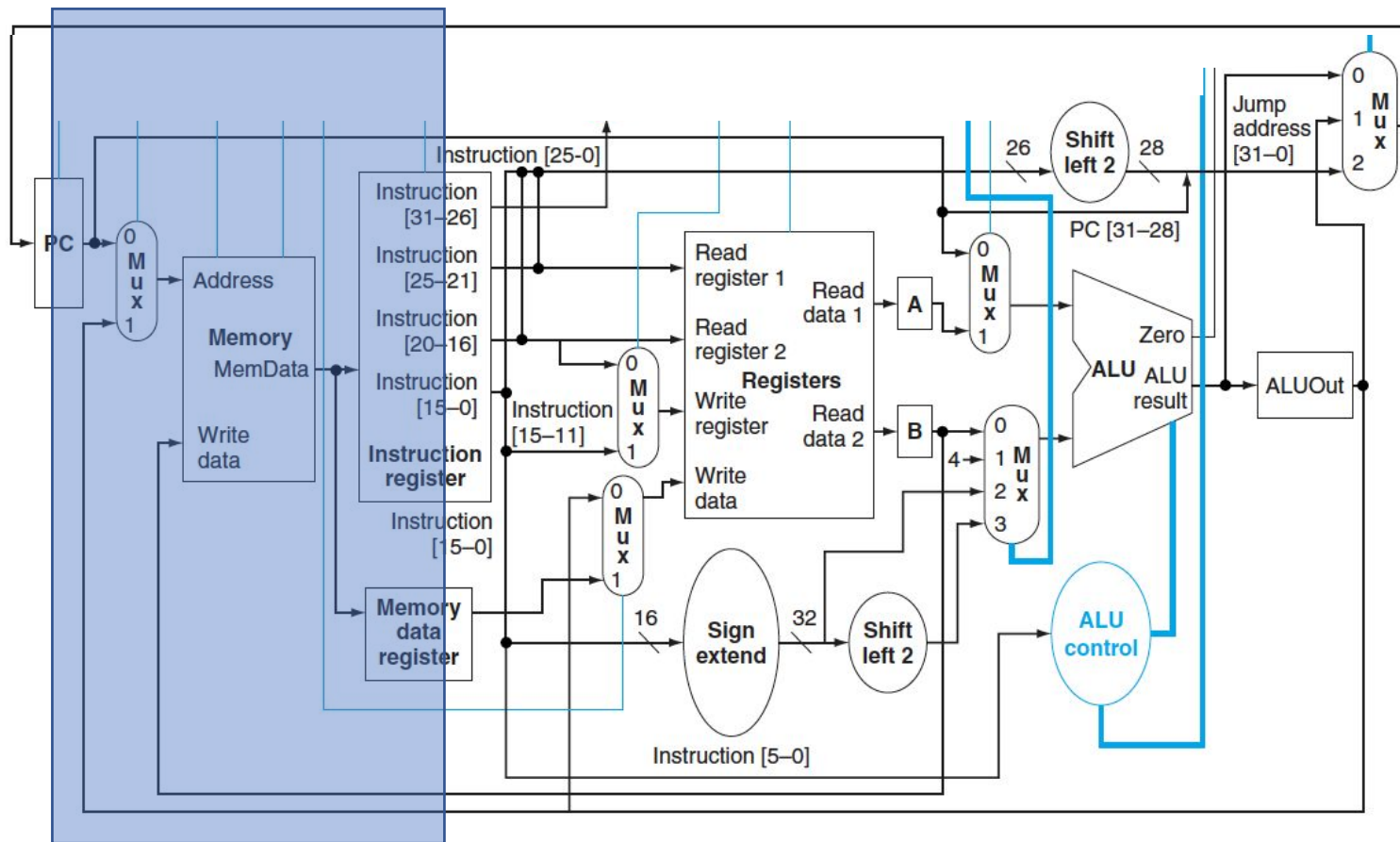
Passo 3 – Acesso a Memória

$ALUOut = A + \text{sign-extend}(IR[15-0]);$



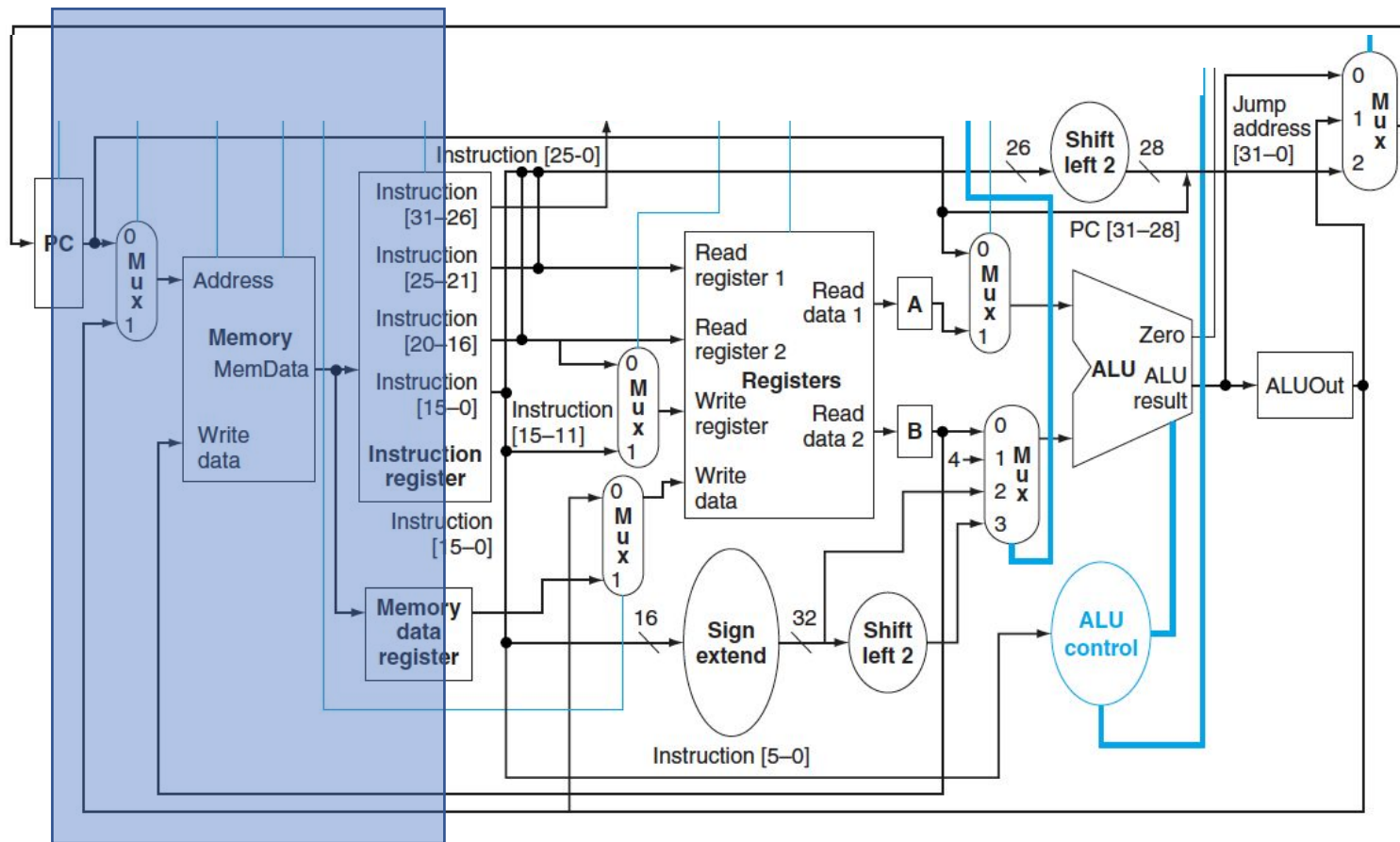
Passo 4 – Acesso a Memória -SW

Memory[ALUOut] = B;



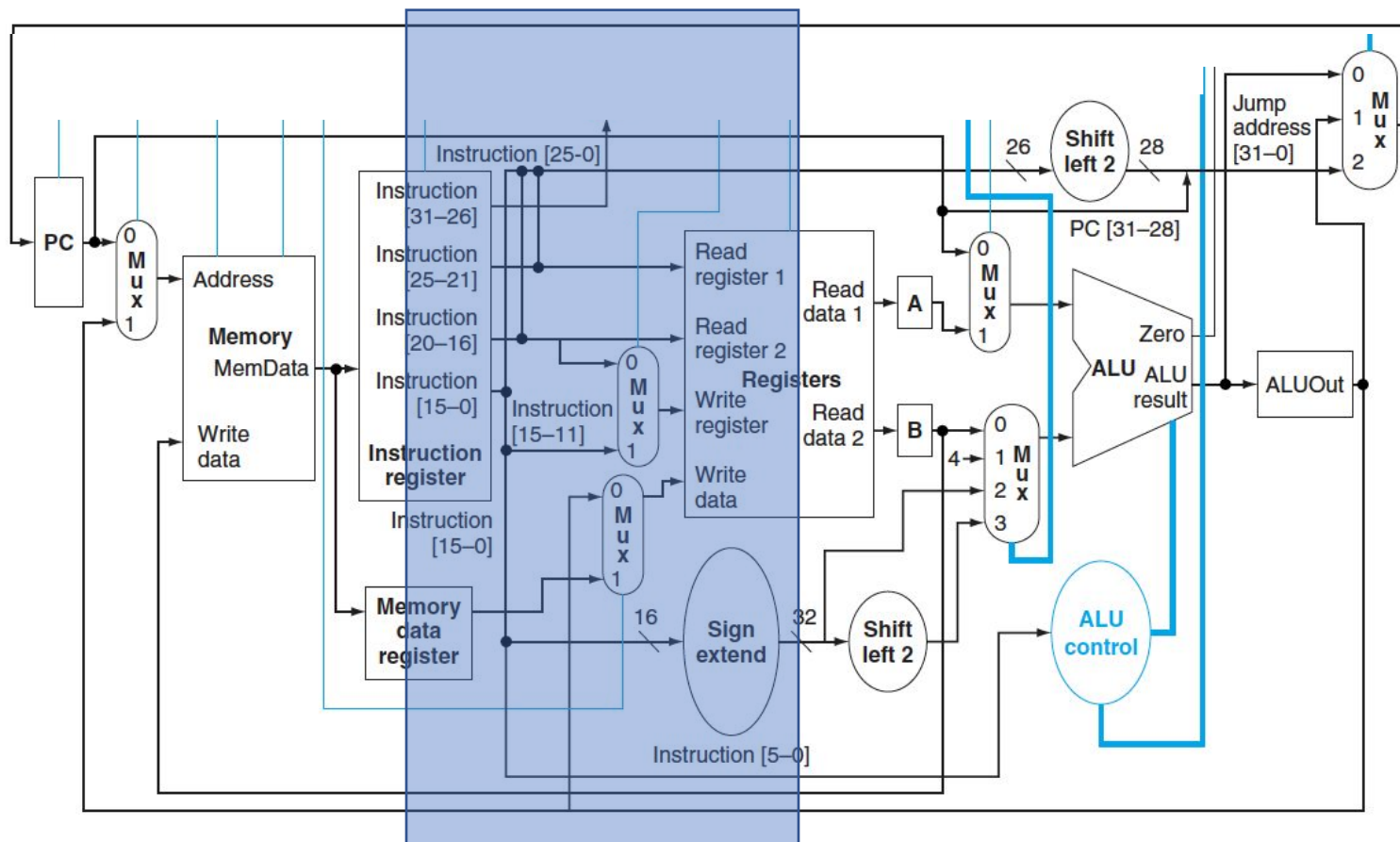
Passo 4 – Acesso a Memória -LW

$MDR = Memory[ALUOut];$



Passo 5 – Acesso a Memória -LW

$\text{Reg}[\text{IR}[20-16]] = \text{MDR};$



Cálculo do Desempenho

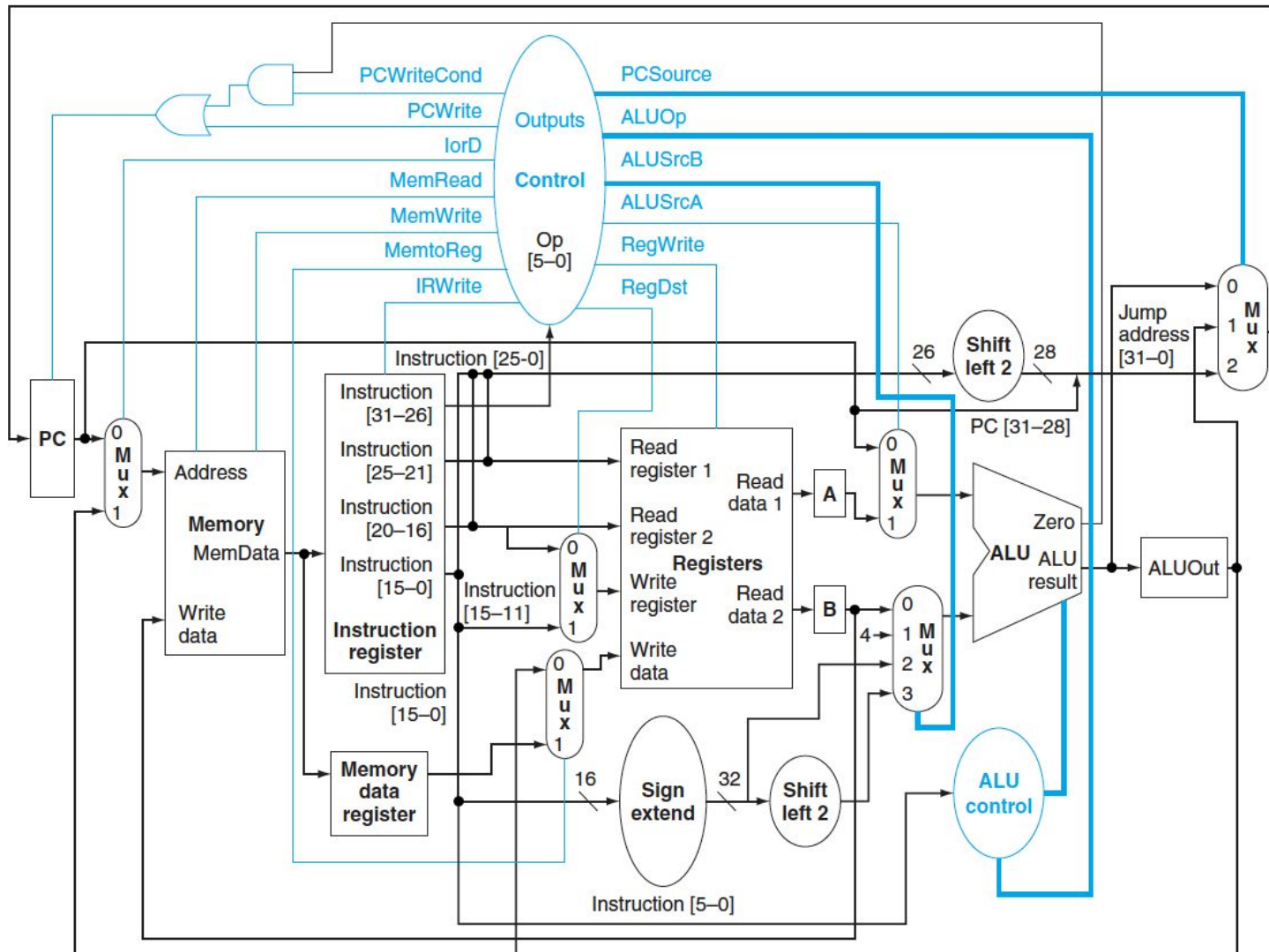
- Considerando
 - 1 ns para acessos à memória
 - 0.5 ns para acessos (escrita ou leitura) ao banco de registradores
 - 0.5 ns para operações na ALU
 - 0 ns para demais blocos (muxs, portas, ...)
- Multiciclo
 - Clock de 1ns no (1 GHz) **Qual é o caminho crítico?**
- Monociclo
 - Clock de 3.5 ns (285 MHz) **Qual é o caminho crítico?**
- Como poderia melhorar o clock do Multiciclo?
 - Poderia ser duplicado (2 GHz) se acessos à memória fossem realizados em 2 ciclos de 0.5 ns cada
- CPI para cada tipo de instrução

	versão 1 GHz	versão 2 GHz
– load:	5	7
– store:	4	6
– tipo R:	4	5
– branch:	3	4
– jump:	3	4

Cálculo do Desempenho

- Mix de instruções do compilador gcc
 - 22% loads, 11% stores, 49% tipo R, 16% branches, 2% jumps
- CPI médio na máquina multi-ciclo
 - $= 0.22 \times 5 + 0.11 \times 4 + 0.49 \times 4 + 0.18 \times 3 = 4.04$ na versão 1 GHz
 - $= 0.22 \times 7 + 0.11 \times 6 + 0.49 \times 5 + 0.18 \times 4 = 5.37$ na versão 2 GHz
- Tempo total de execução do programa gcc no MIPS mono-ciclo
 - $= N \times \text{CPI} \times \text{período do clock}$
 - $= N \times 1 \times 3.5 \text{ ns} = 3.5 N \times 10^{-9}$
- Tempo total de execução do programa gcc no MIPS multi-ciclo
 - $= N \times \text{CPI médio} \times \text{período do clock}$
 - $= N \times 4.04 \times 1 \text{ ns} = 4.04 N \times 10^{-9}$ na versão 1 GHz
 - $= N \times 5.37 \times 0.5 \text{ ns} = 2.685 N \times 10^{-9}$ na versão 2 GHz

Multi-Ciclo Completo



Sinais de controle

MemRead = 1	Ler conteúdo da memória
MemWrite = 1	Escrever conteúdo na memória
ALUSelA = 0	PC é primeiro operando para ALU
ALUSelA = 1	Registrador A é primeiro operando para ALU
ALUSelB = 00	Registrador B é segundo operando para ALU
ALUSelB = 01	Constante = 4 é segundo operando para ALU
ALUSelB = 10	Deslocamento (estendido p/ 32 bits) é segundo operando para ALU
ALUSelB = 11	Deslocamento (estendido p/ 32 bits e deslocado 2 bits p/ esq.) é 2º operando p/ ALU
RegDst = 0	Registrador a ser escrito é especificado pelo campo rt
RegDst = 1	Registrador a ser escrito é especificado pelo campo rd
RegWrite = 1	Escrever conteúdo em registrador
MemtoReg = 0	Valor a ser escrito em registrador vem do registrador da saída da ALU (ALUout)
MemtoReg = 1	Valor a ser escrito em registrador vem do registrador de dados da memória (MDR)
lorD = 0	Endereço em memória vem do PC
lorD = 1	Endereço em memória vem da ALU
IRWrite = 1	Armazenar instrução no IR

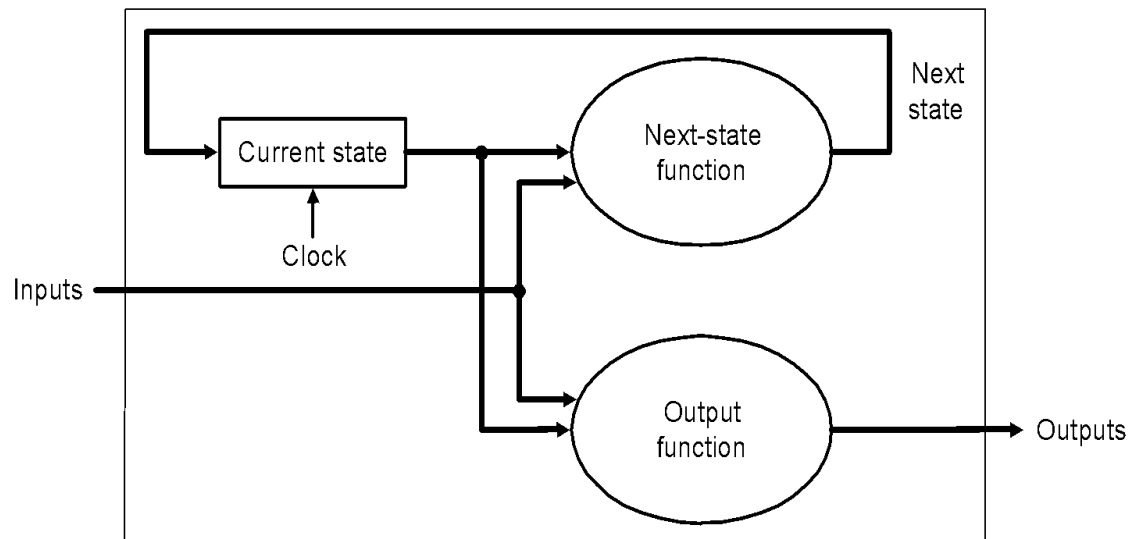
Sinais de controle para PC

PCWrite = 1	PC é carregado; fonte é controlada por
PCSource PCWriteCond = 1 ligada	PC é carregado se saída Zero da ALU está

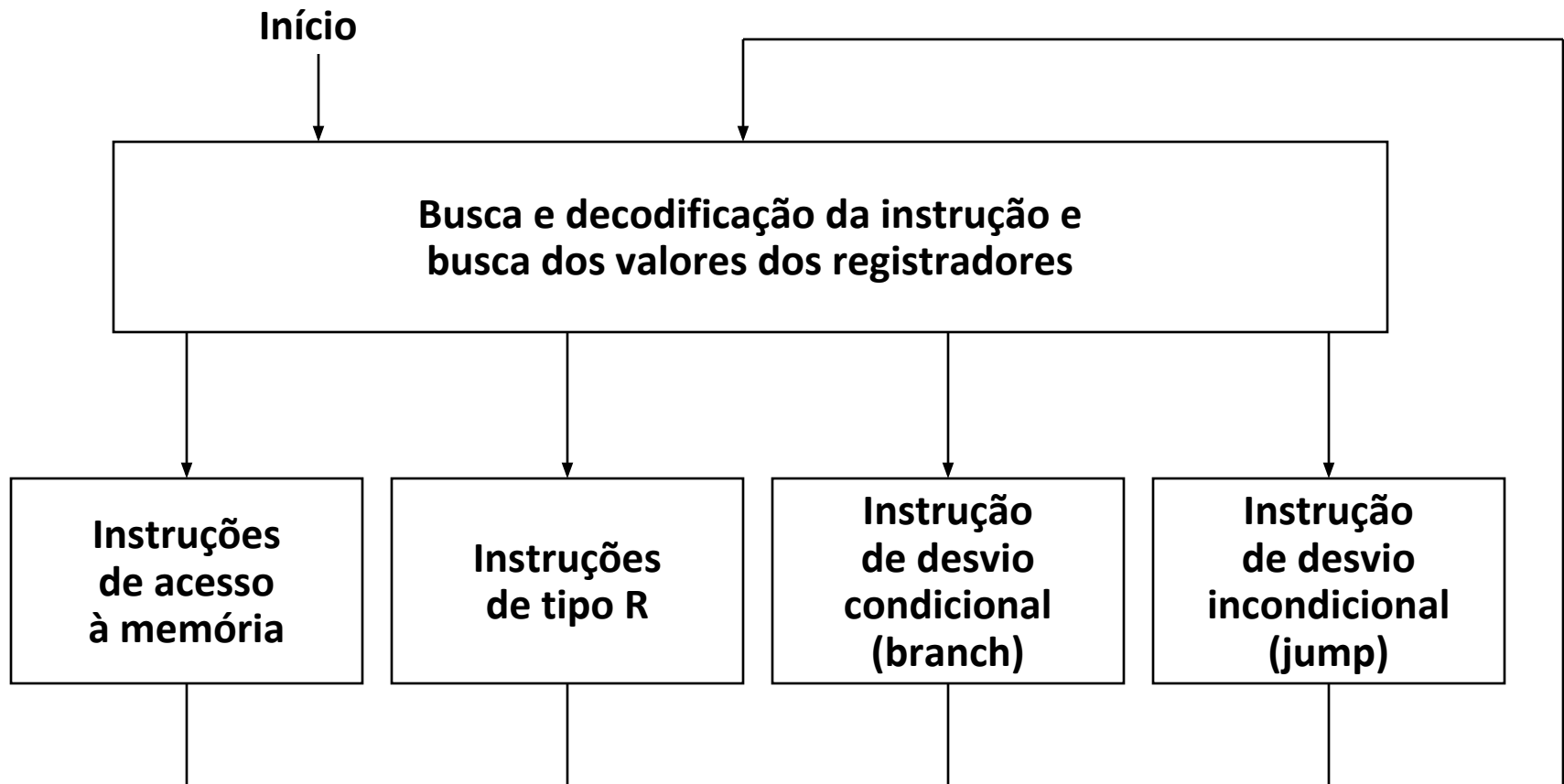
PCSource	00	Saída da ALU é enviada para o PC
	01	Conteúdo do registrador ALUout é enviado para o PC
	10	Endereço de destino do Jump enviado para o PC

Máquina de estados finitos

- Finite state machines (FSMs):
 - Um conjunto de estados
 - Uma função de próximo estado
 - Determinada pelo estado corrente e entrada
 - Uma função de saída
 - Determinada pelo estado corrente (somente, ou com a entrada)

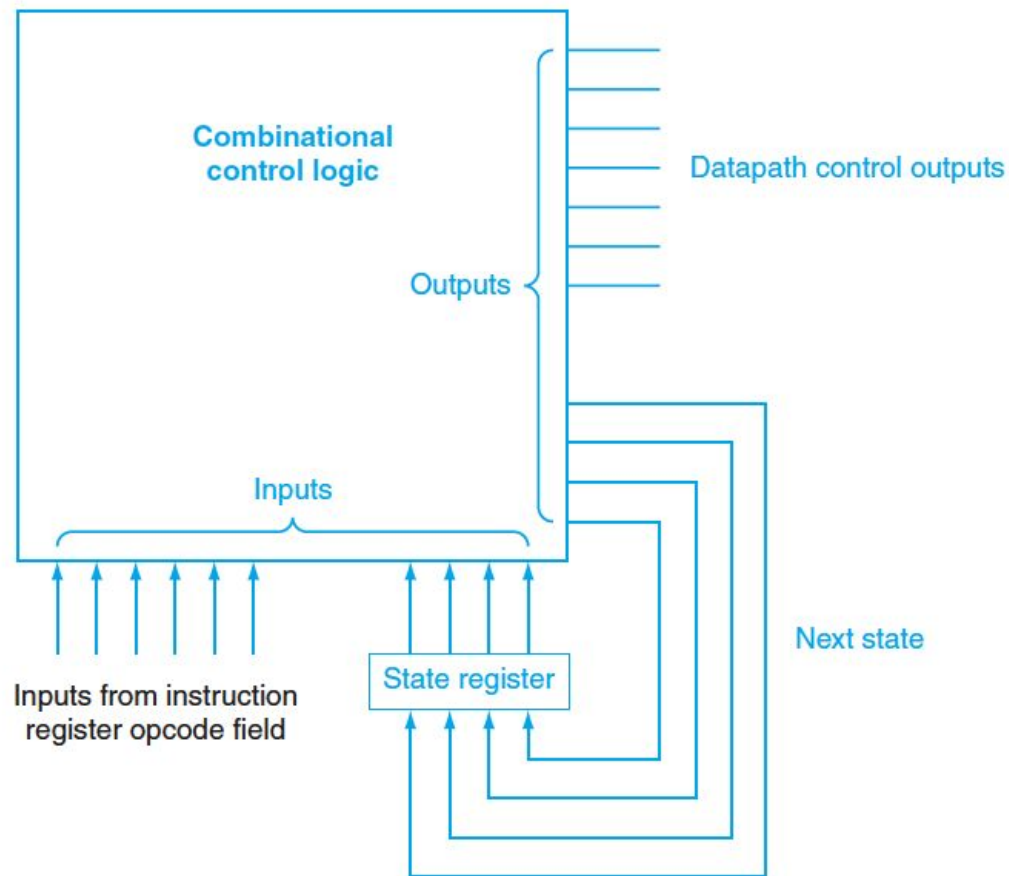


Máquina de estados finitos

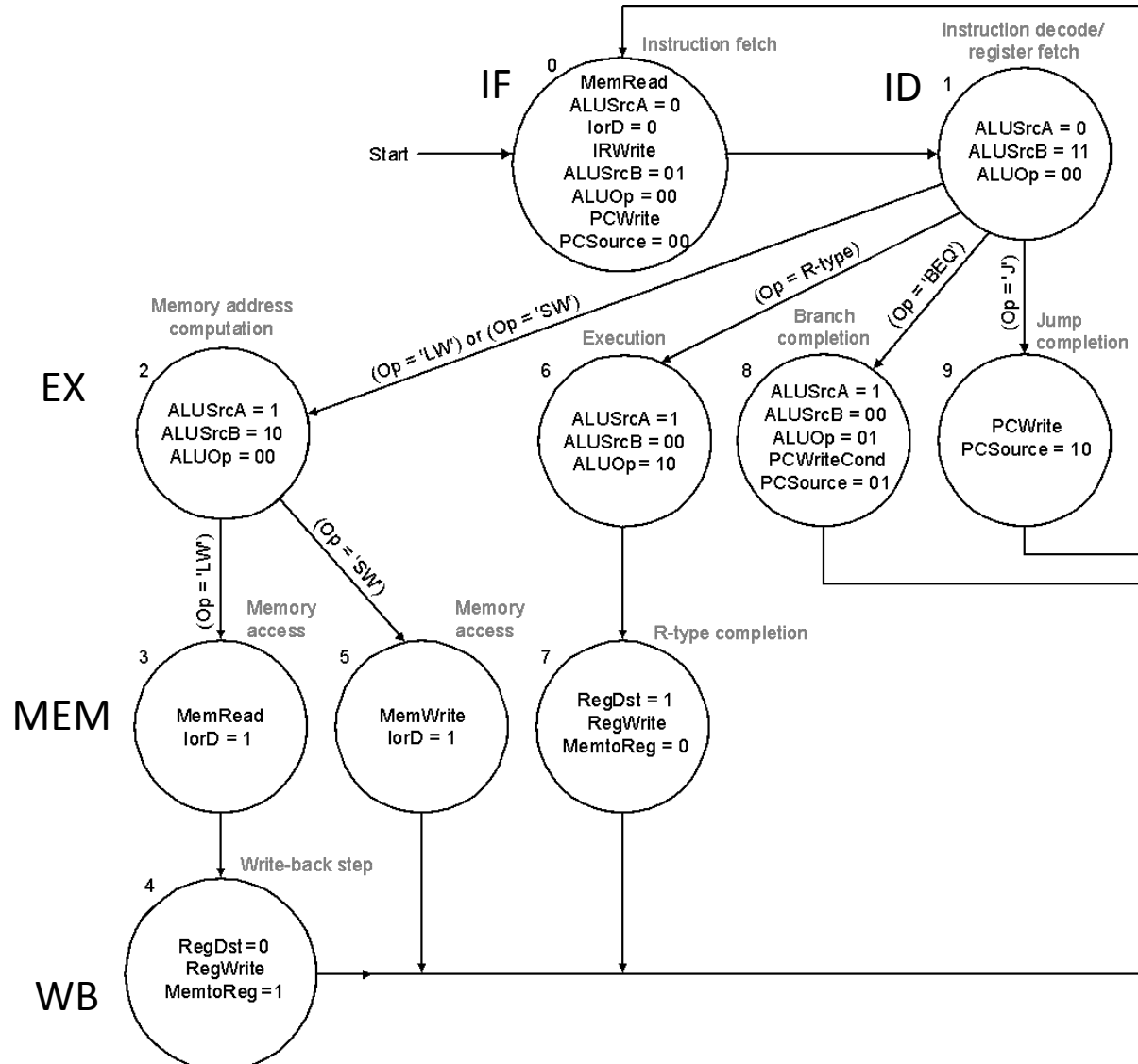


Máquina de estados finitos

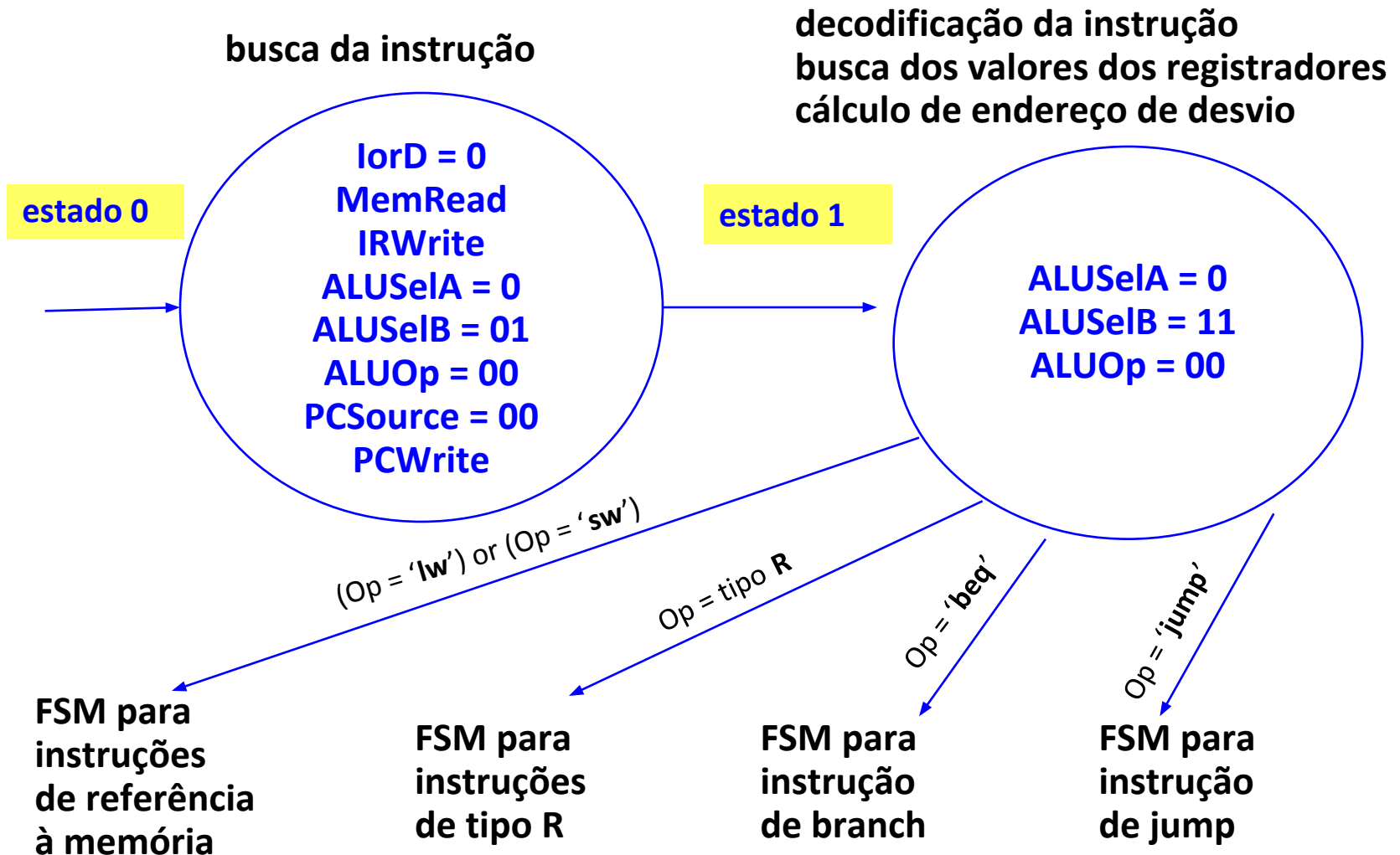
- Máquina estado do MIPS Multiciclo



Máquina de Estados Completa



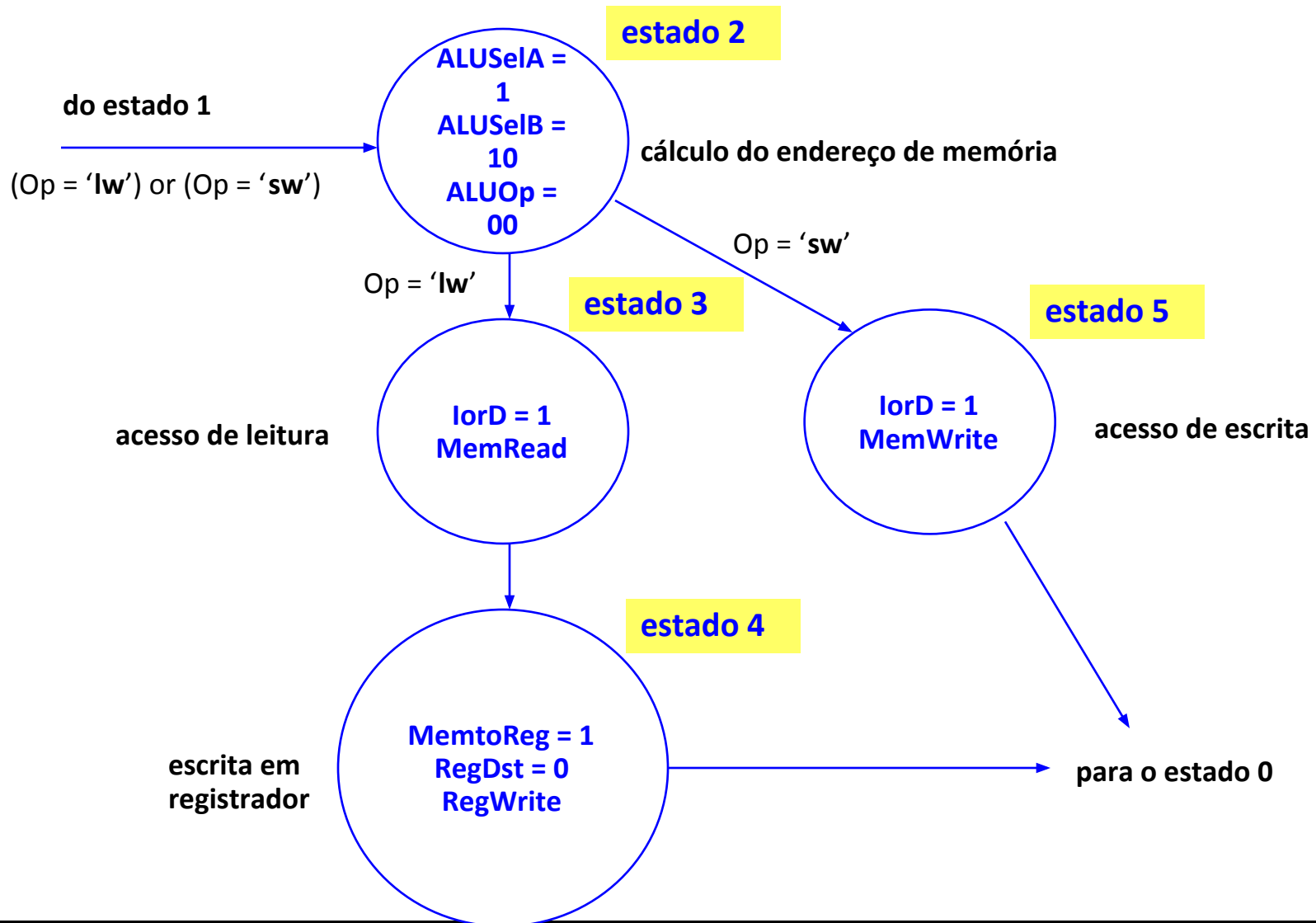
Busca e decodificação de instruções



Busca e decodificação de instruções

- Estado 0 – busca da instrução
 - $lorD = 0$: PC é fonte de endereço para a memória
 - MemRead: ler instrução da memória
 - IRWrite: escrever instrução no IR
 - $ALUSelA = 0$: PC é primeiro operando para ALU
 - $ALUSelB = 01$: constante 4 é segundo operando para ALU
 - $ALUOp = 00$: ALU soma $PC + 4$
 - $PCSource = 00$: Saída da ALU é enviada para o PC
 - PCWrite: PC é escrito com $PC + 4$
- Estado 1 – decodificação da instrução, busca dos valores dos registradores, cálculo do endereço de desvio condicional (branch)
 - $ALUSelA = 0$: PC é primeiro operando para ALU
 - $ALUSelB = 10$: deslocamento é segundo operando para ALU
 - $ALUOp = 00$: ALU soma $PC + \text{deslocamento}$

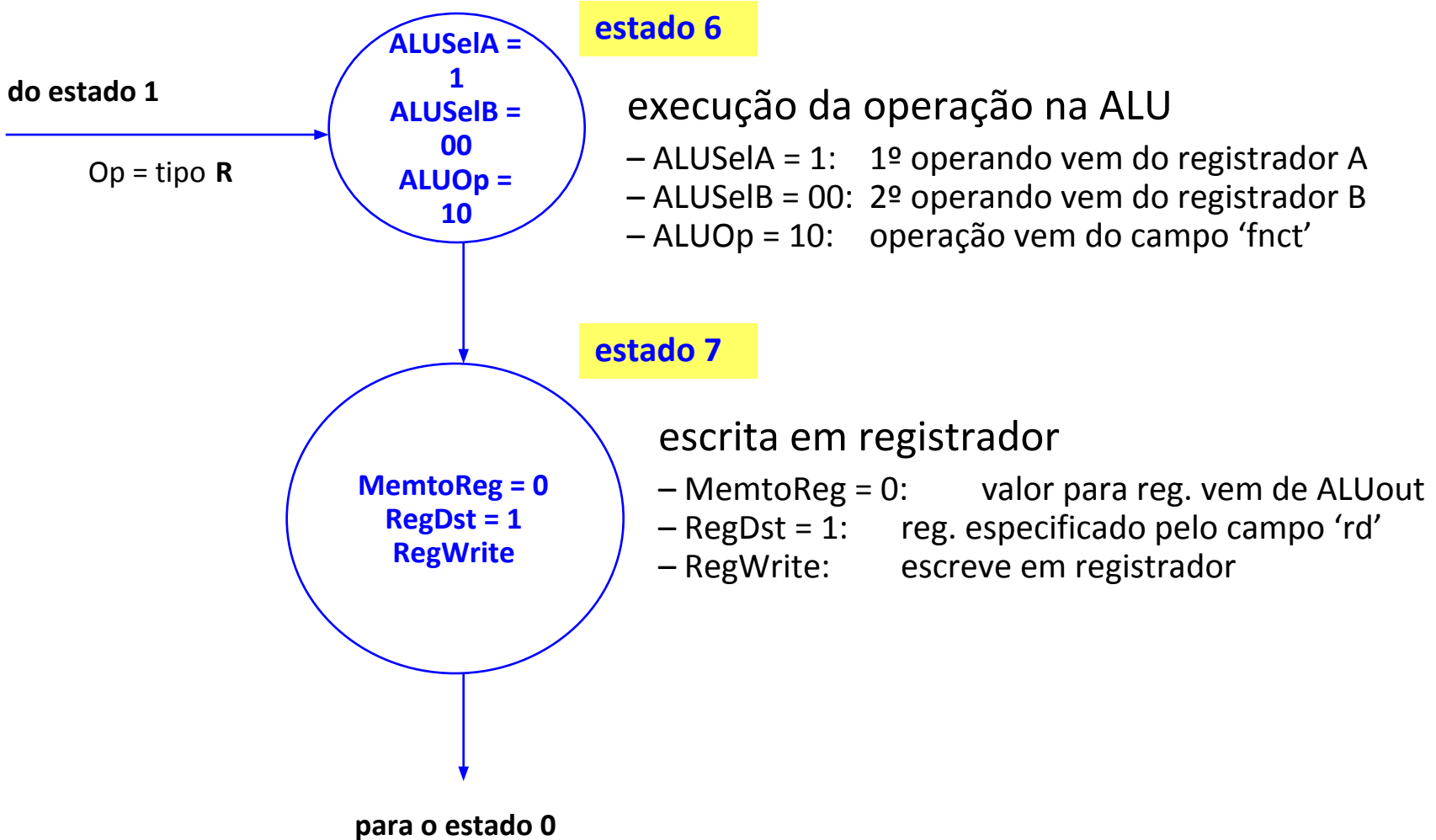
Instruções de Acesso à Memória



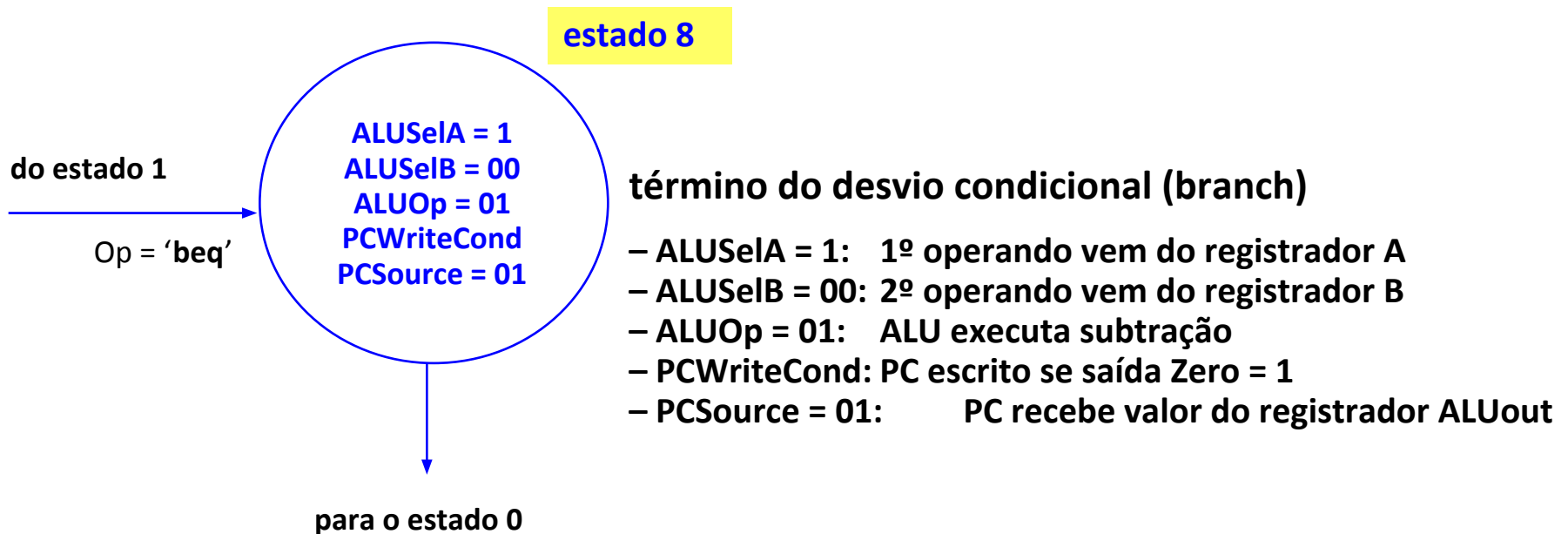
Instruções de Acesso à Memória

- Estado 2 – cálculo do endereço efetivo
 - ALUSelA = 1: registrador base é primeiro operando para ALU
 - ALUSelB = 10: deslocamento é segundo operando para ALU
 - ALUOp = 00: ALU soma base + deslocamento
- Estado 3 – acesso de leitura na memória
 - lorD = 1: endereço de memória vem da ALU
 - MemRead: leitura na memória
- Estado 4 – escrita em registrador
 - MemtoReg = 1: valor para registrador vem da memória
 - RegDst = 0: registrador a ser escrito especificado pelo campo rd
 - RegWrite: escrita em registrador
- Estado 5 – acesso de escrita na memória
 - lorD = 1: endereço de memória vem da ALU
 - MemWrite: escrita na memória

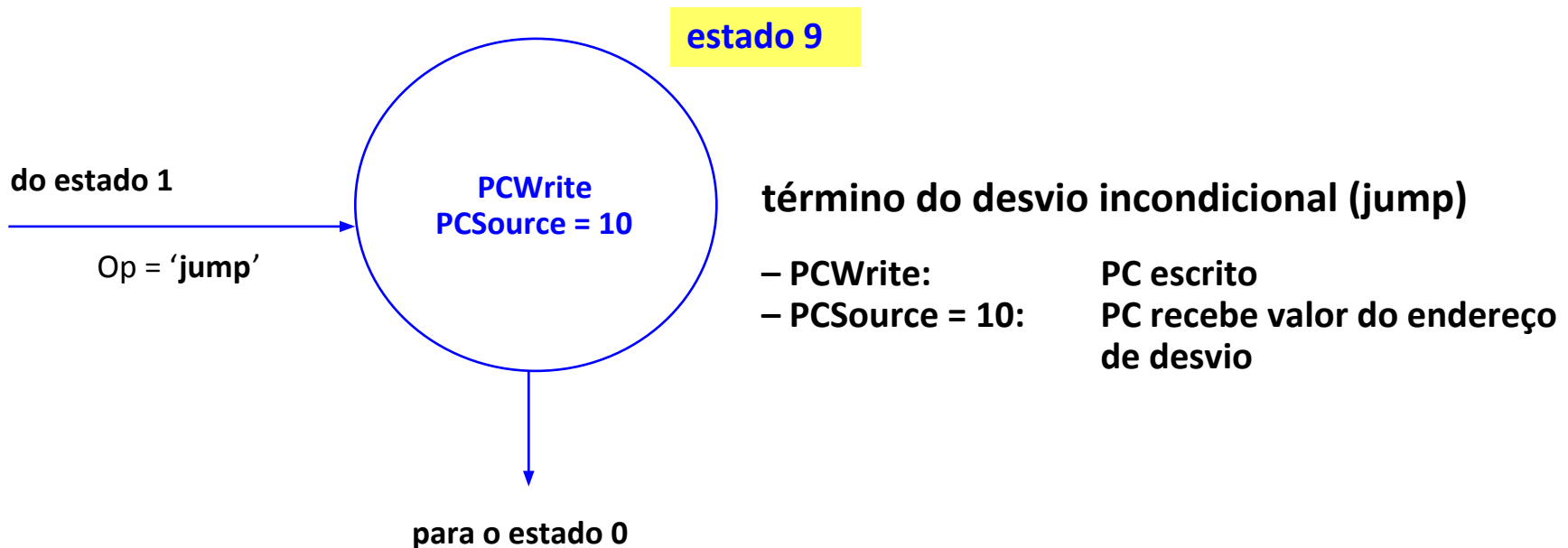
Instruções tipo R



Instrução Branch – Desvio Condicional



Instrução Jump – Desvio Incondicional



Exceções

- Eventos inesperado que mudam o fluxo normal da execução de instruções do programa
- Interrupções vs Exceções
 - Interrupções são eventos advindos de fora do processador
 - Chamadas de I/O
 - Exceções são eventos internos do processador
 - Overflow na ALU, Instrução desconhecida
- No MIPS

Type of event	From where?	MIPS terminology
I/O device request	External	Interrupt
Invoke the operating system from user program	Internal	Exception
Arithmetic overflow	Internal	Exception
Using an undefined instruction	Internal	Exception
Hardware malfunctions	Either	Exception or interrupt

Manipulando Exceções

- Vamos cobrir dois casos de exceções:
 - Overflow aritmético
 - Instrução indefinida
- Passos para manipulação:
 - Salvar o endereço da instrução que gerou a exceção no registrador EPC ($EPC \leftarrow PC$?)
 - Transferir o controle para o Sistema Operacional (SO) tratar a exceção
 - Ir para um endereço de memória que contém a função que trata a exceção ($PC \leftarrow$ endereço da função)
 - Após tratar a exceção, o SO pode tomar decisões:
 - Terminar o programa que gerou a exceção
 - Continuar o programa, retornando do endereço salvo no EPC

Manipulando Exceções

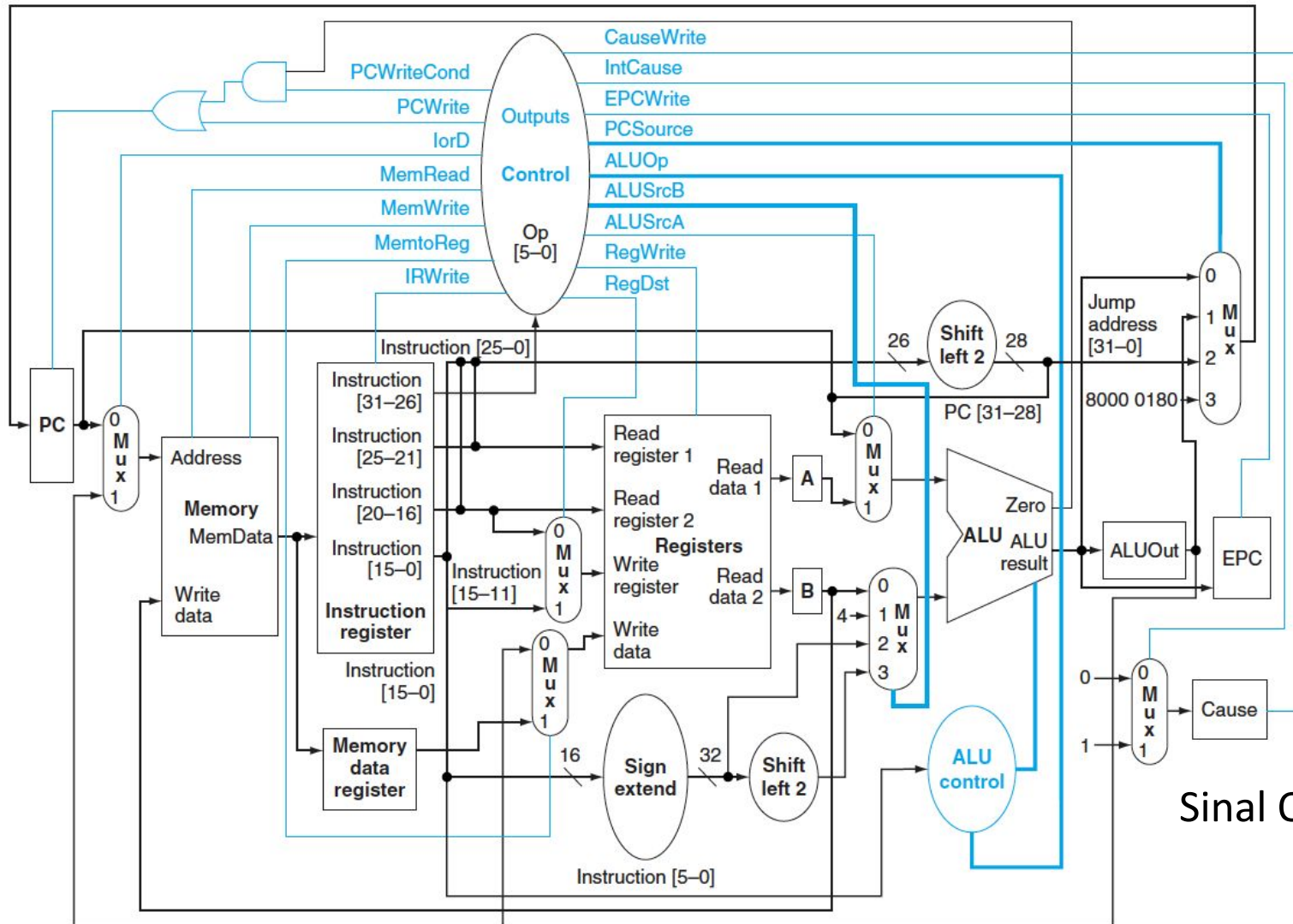
- Comunicando o SO sobre qual exceção foi gerada:
 - **Registrador de *status*** (no MIPS, *Cause Register*)
 - SO deve decodificar o campo referente aos bits de interrupção para saber qual é a exceção
 - **Interrupções vetoradas** – o processador salta para um endereço pré definido de cada exceção
 - Espaço de 32 bytes(00-20), 8 instruções, que é designado para o SO colocar uma função (SW) que identifique a razão da interrupção e realizar processamento para manipular a exceção

Exception type	Exception vector address (in hex)
Undefined instruction	C000 0000 _{hex}
Arithmetic overflow	C000 0020 _{hex}

Implementando Exceções no MIPS

- Implementação baseada no **Registrador de Status**, o SO deverá identificar a exceção através do bit menos significativo deste registrador:
 - **(bit =0) Instrução Indefinida** – é detectada no estado 1 (2 ciclo da instrução) da máquina de estados, quando a função de próximo estado não identifica o OPCode
 - **(bit=1) Overflow Aritmético** – é detectada no estado 7 (4 ciclo da instrução aritmética) por um sinal de saída na ULA específico para informar overflow
- Para ambas, o PC deve receber o endereço da função que irá decodificar o bit e identificar qual interrupção ocorreu
 - $PC \leftarrow 0x8000\ 0180$

Implementando Exceções no MIPS



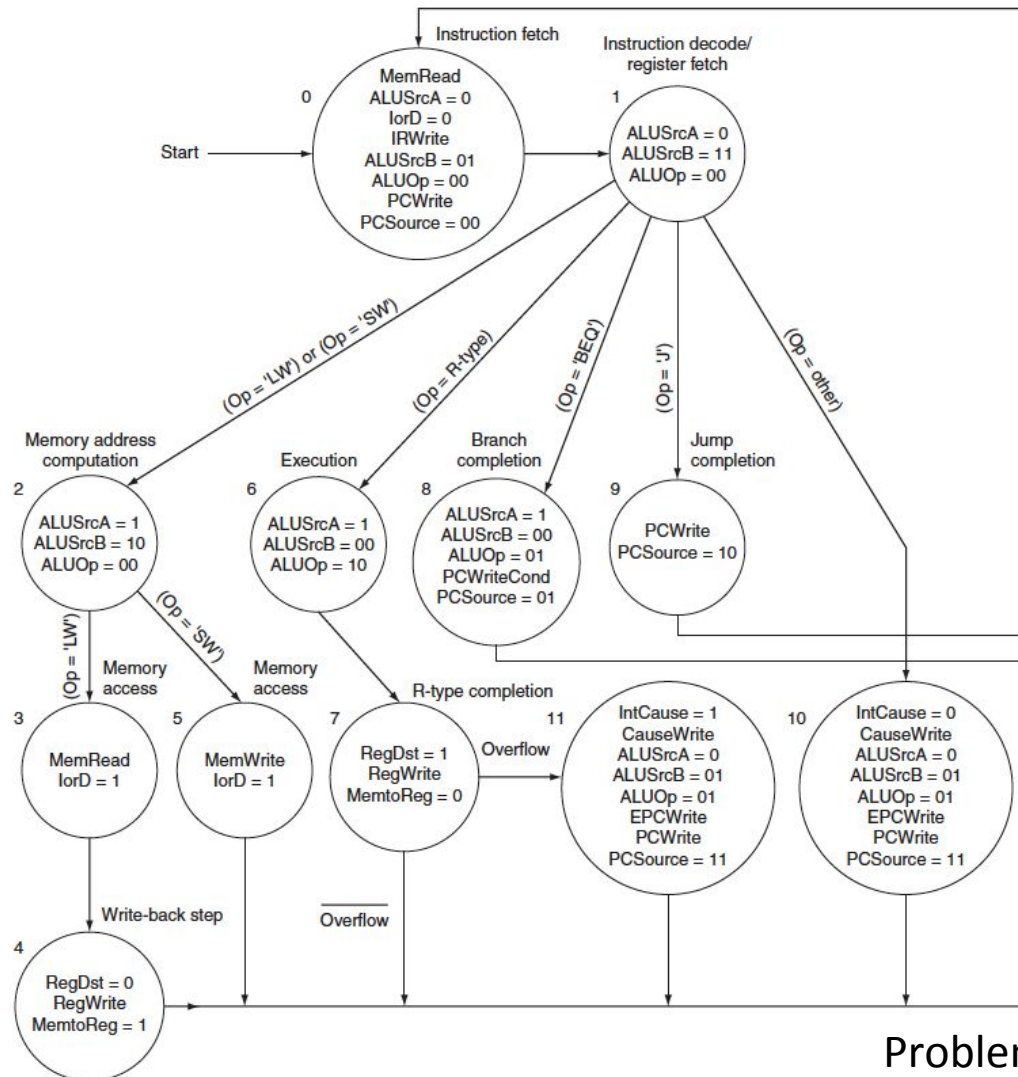
Sinal Overflow?

Implementando Exceções no MIPS



- Para implementar o Overflow deve-se:
 - Adicionar um registrador de um bit para guardar o sinal de overflow da ULA
 - Enviar a saída do registrador para o controle

Implementando Exceções no MIPS



Problema com Overflow?

5.2 [10] <§5.4> Describe the effect that a single stuck-at-0 fault (i.e., regardless of what it should be, the signal is always 0) would have for the signals shown below, in the single-cycle datapath in Figure 5.17 on page 307. Which instructions, if any, will not work correctly? Explain why.

Consider each of the following faults separately:

- a. $\text{RegWrite} = 0$
- b. $\text{ALUOp0} = 0$
- c. $\text{ALUOp1} = 0$
- d. $\text{Branch} = 0$
- e. $\text{MemRead} = 0$
- f. $\text{MemWrite} = 0$

5.3 [5] <§5.4> This exercise is similar to Exercise 5.2, but this time consider stuck-at-1 faults (the signal is always 1).

5.8 [15] <§5.4> We wish to add the instruction `jr` (jump register) to the single-cycle datapath described in this chapter. Add any necessary datapaths and control signals to the single-cycle datapath of Figure 5.17 on page 307 and show the necessary additions to Figure 5.18 on page 308. You can photocopy these figures to make it faster to show the additions.

Exercícios

5.11 [20] <\$5.4> This question is similar to Exercise 5.8 except that we wish to add a variant of the `lw` (load word) instruction, which increments the index register after loading word from memory. This instruction (`lw_inc`) corresponds to the following two instructions:

```
lw    $rs,L($rt)
addi  $rt,$rt,1
```

5.12 [5] <\$5.4> Explain why it is not possible to modify the single-cycle implementation to implement the load with increment instruction described in Exercise 5.12 without modifying the register file.

5.13 [7] <\$5.4> Consider the single-cycle datapath in Figure 5.17. A friend is proposing to modify this single-cycle datapath by eliminating the control signal `MemtoReg`. The multiplexor that has `MemtoReg` as an input will instead use either the `ALUSrc` or the `MemRead` control signal. Will your friend's modification work? Can one of the two signals (`MemRead` and `ALUSrc`) substitute for the other? Explain.

5.14 [10] <\$5.4> MIPS chooses to simplify the structure of its instructions. The way we implement complex instructions through the use of MIPS instructions is to decompose such complex instructions into multiple simpler MIPS ones. Show how MIPS can implement the instruction `swap $rs, $rt`, which swaps the contents of registers `$rs` and `$rt`. Consider the case in which there is an available register that may be destroyed as well as the case in which no such register exists.

If the implementation of this instruction in hardware will increase the clock period of a single-instruction implementation by 10%, what percentage of swap operations in the instruction mix would recommend implementing it in hardware?

Exercícios

5.29 [5] <§5.5> This exercise is similar to Exercise 5.2, but this time consider the effect that the stuck-at-0 faults would have on the *multiple-cycle* datapath in Figure 5.27. Consider each of the following faults:

- a. RegWrite = 0
- b. MemRead = 0
- c. MemWrite = 0
- d. IRWrite = 0
- e. PCWrite = 0
- f. PCWriteCond = 0.

5.30 [5] <§5.5> This exercise is similar to Exercise 5.29, but this time consider stuck-at-1 faults (the signal is always 1).

Exercícios

5.49 [30] <§5.6> We wish to add the instruction `eret` (exception return) to the multicycle datapath described in this chapter. A primary task of the `eret` instruction is to reload the PC with the return address at which an exception, or error trap occurred. Suppose that if the processor is serving an error trap, then the PC has to be loaded from a register `ErrorPC`. Otherwise the processor is serving an exception) the PC has to be loaded from `EPC`. Suppose that there is a bit in the cause register called `trap` to encode an error trap when it occurs and to save the PC in the `ErrorPC` register. Add any necessary datapaths and control signals to the multicycle datapath of Figure 5.39 on page 344 to accommodate the trap/exception call and return, and show the necessary modifications to the finite state machine of Figure 5.40 on page 345 to implement the `eret` instruction. You can photocopy the figures to make it easier to show your modifications.

5.50 [6] <§5.6> Exceptions occur when a control flow change is required to handle an unexpected event in the processor. How can the cause and the instruction that caused the exception, be represented by the hardware in a MIPS machine? Give two examples for conditions that a processor can handle by restarting execution of instructions after handling the exception, and two others for exceptions that lead to program termination.

Exercícios

1. Se um programa tem 20% acesso a memória, 30% desvios e 50% aritméticas, e supondo que o custo de cada grupo de instruções em ciclos é de 5, 3 e 2, respectivamente, qual o CPI médio?
2. No exemplo acima, conseguiu-se reduzir o custo de desvios para 1. Qual o novo CPI?
3. Nesta mesma máquina, o preço a pagar para reduzir o custo do desvio foi um aumento em 20% no ciclo de relógio. Qual o ganho em tempo final?

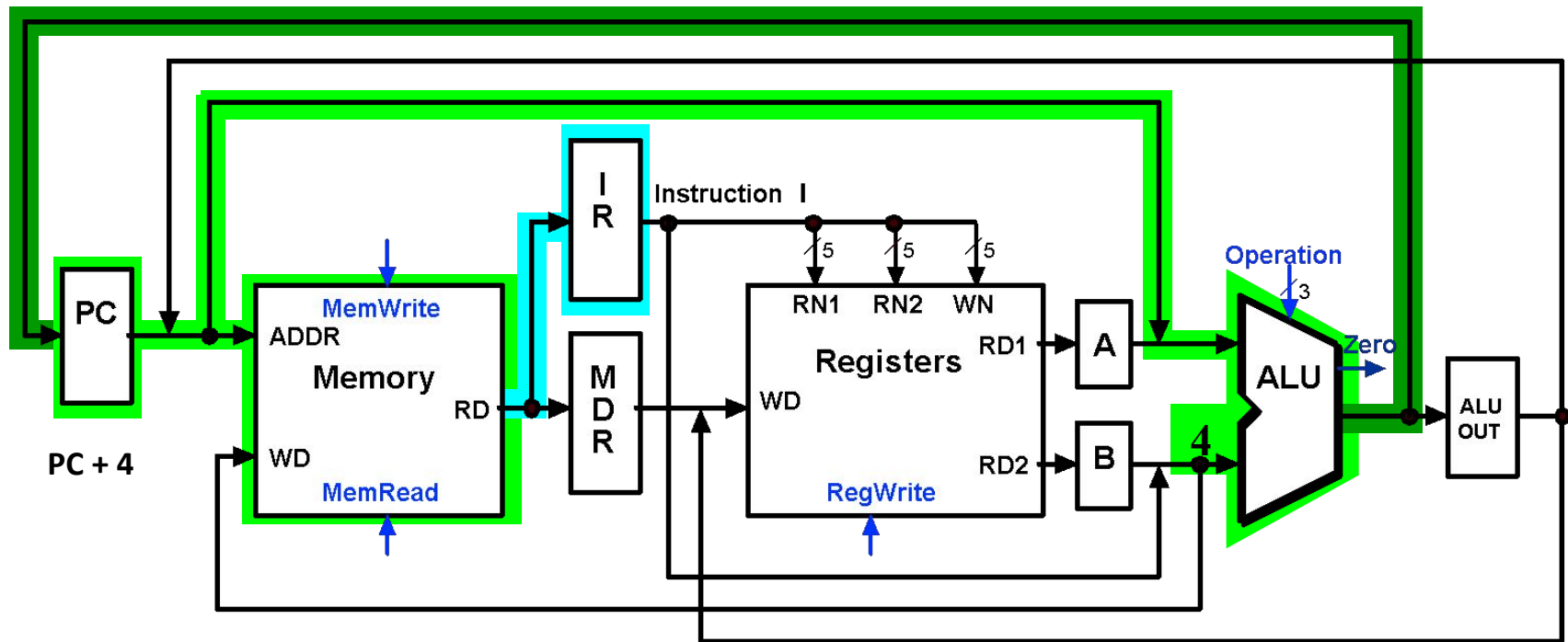
Para ler



- Organização e Projeto de Computadores (Patterson & Hennessy)
 - Segunda Edição:
 - Cap. 5.4
 - Terceira Edição
 - Cap. 5.5

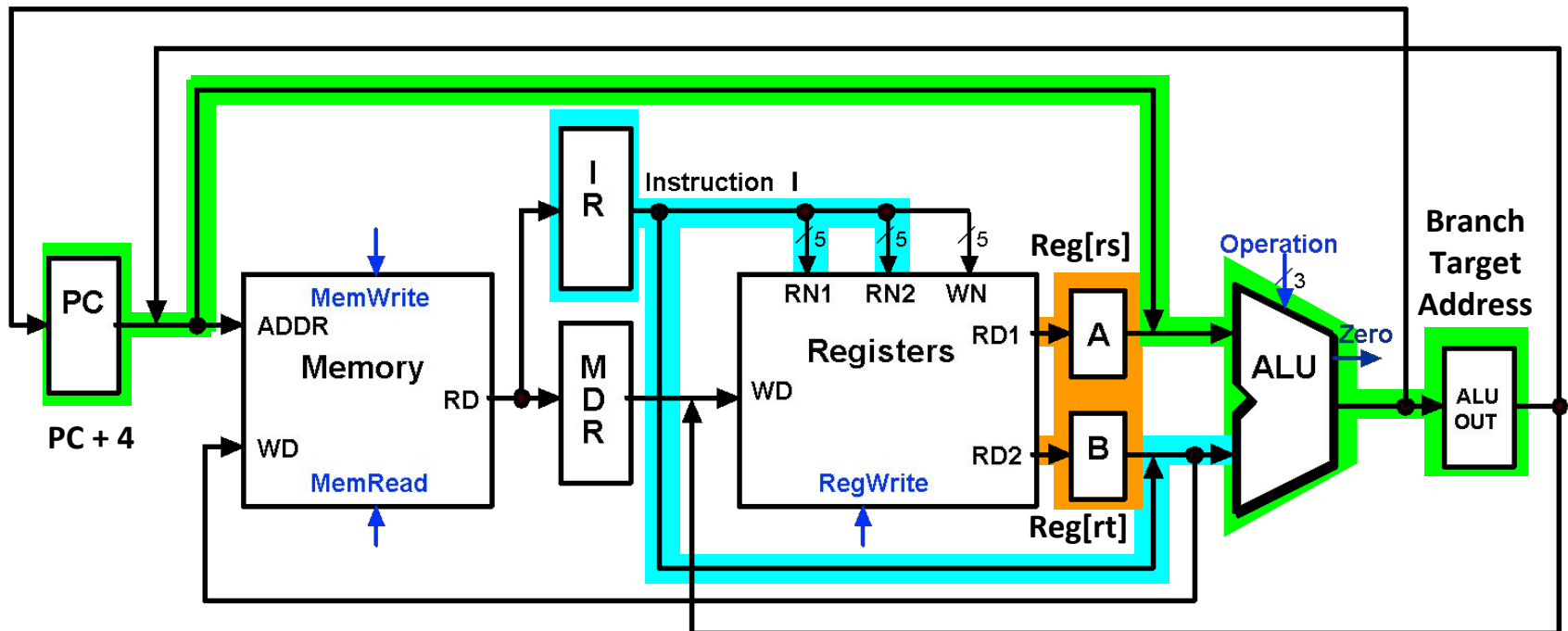
Load Com Pós Incremento

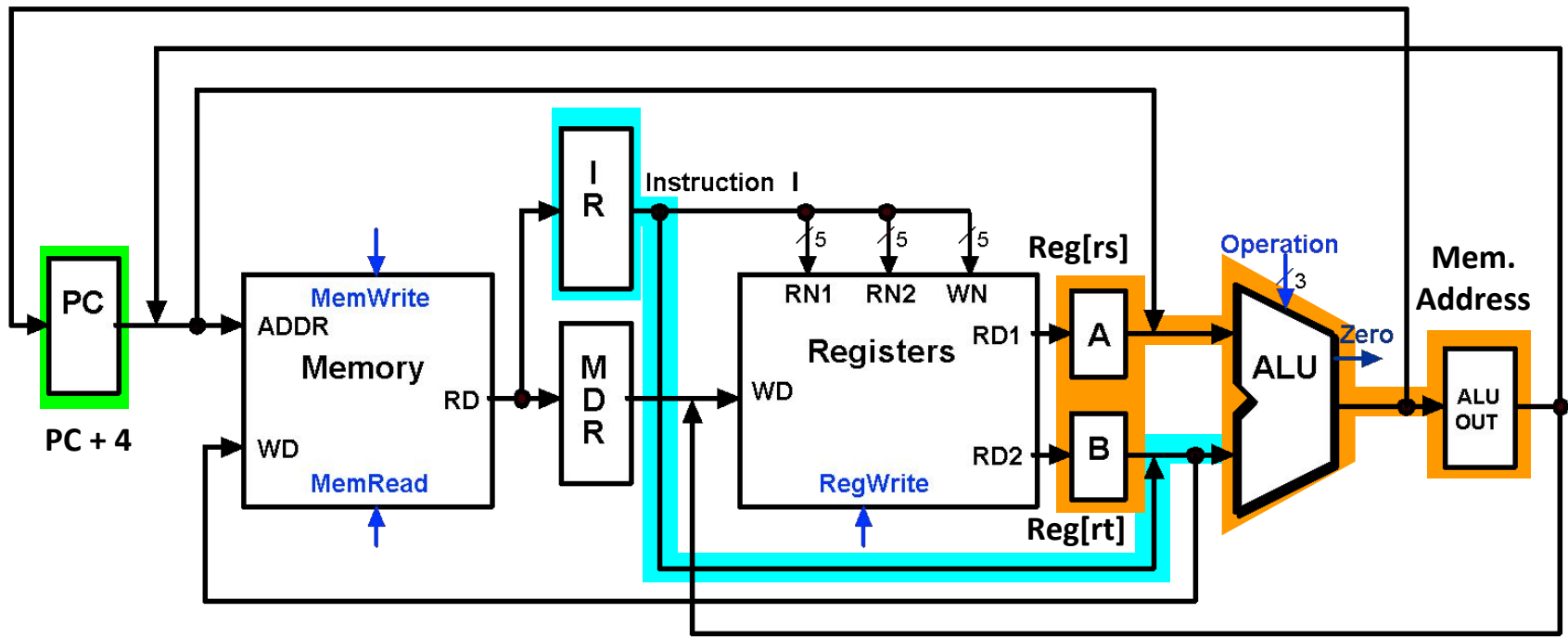

```
IR = Memory[PC];  
PC = PC + 4;
```



```

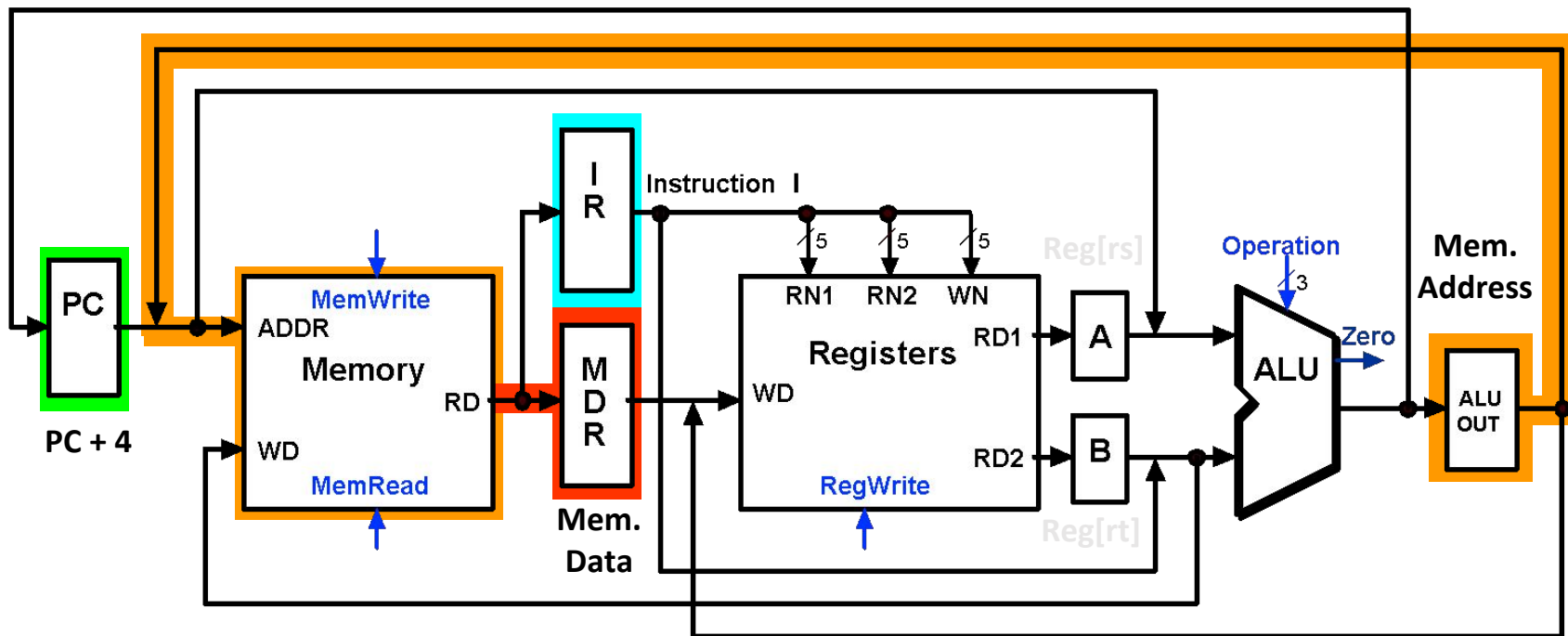
A = Reg[IR[25-21]];      (A = Reg[rs])
B = Reg[IR[20-15]];      (B = Reg[rt])
ALUOut = (PC + sign-extend(IR[15-0]) << 2)
    
```



$$\text{ALUOut} = A + \text{sign-extend}(\text{IR}[15-0]);$$


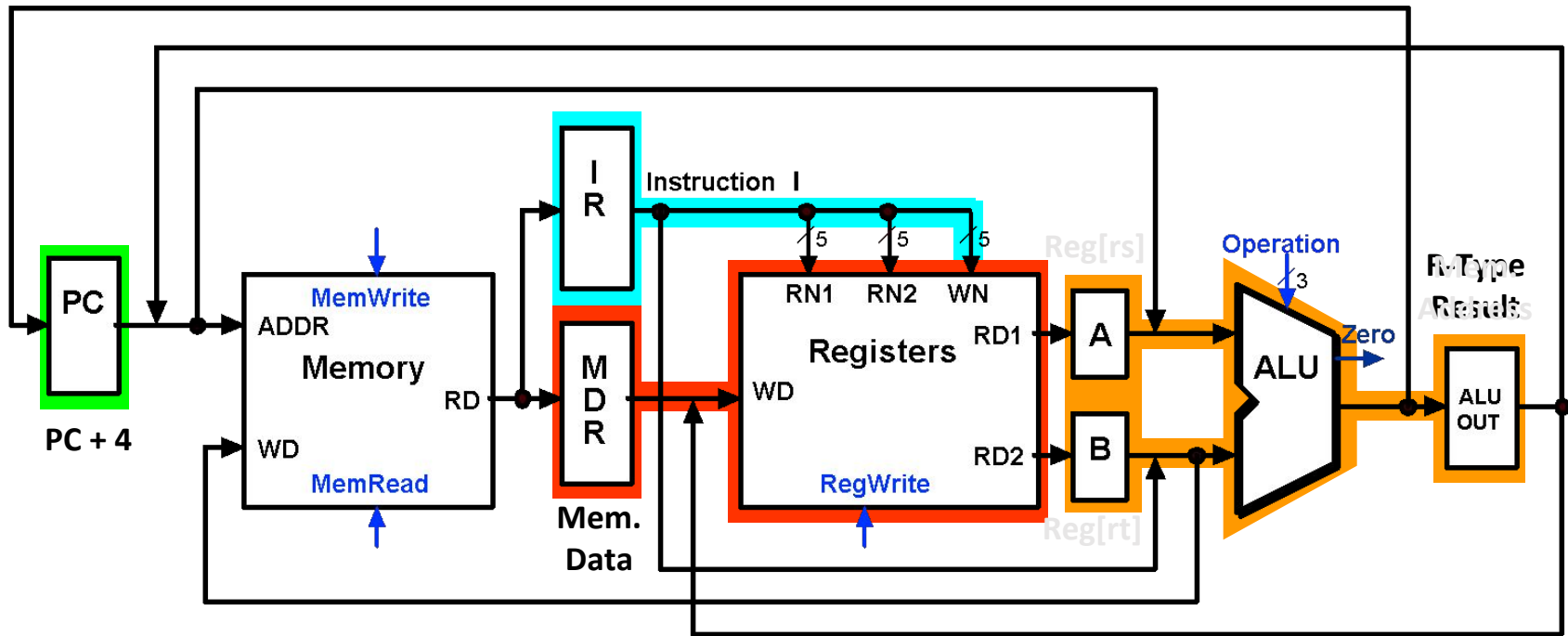
LWPI

`MDR = Memory[ALUOut];`



LWPI

```
Reg[IR[20-16]] = MDR;  
ALUOut = A + 4;
```



LWPI

`Reg[IR[25-21]] = ALUOut;`

